

Лекции 1-2

1. Архитектура СУБД

Архитектура ANSI-SPARC

Предложена в 1975 г. подкомитетом SPARC (Standards Planning And Requirements Committee) ANSI.

Архитектура СУБД включает в себя 3 уровня:

- Внешний (пользовательский).
- Промежуточный (концептуальный).
- Внутренний (физический).

Почти все современные СУБД соответствуют принципам ANSI-SPARC.

СУБД PostgreSQL

- Свободно-распространяемая СУБД на основе языка SQL.
- По классификации — *объектно-реляционная клиент-серверная СУБД*.
- Разработка (Postgres) началась в 1986 году.
- Postgres95 — введен язык SQL.
- С 1996 — PostgreSQL.
- Актуальная версия — 14.1.

Структуры СУБД

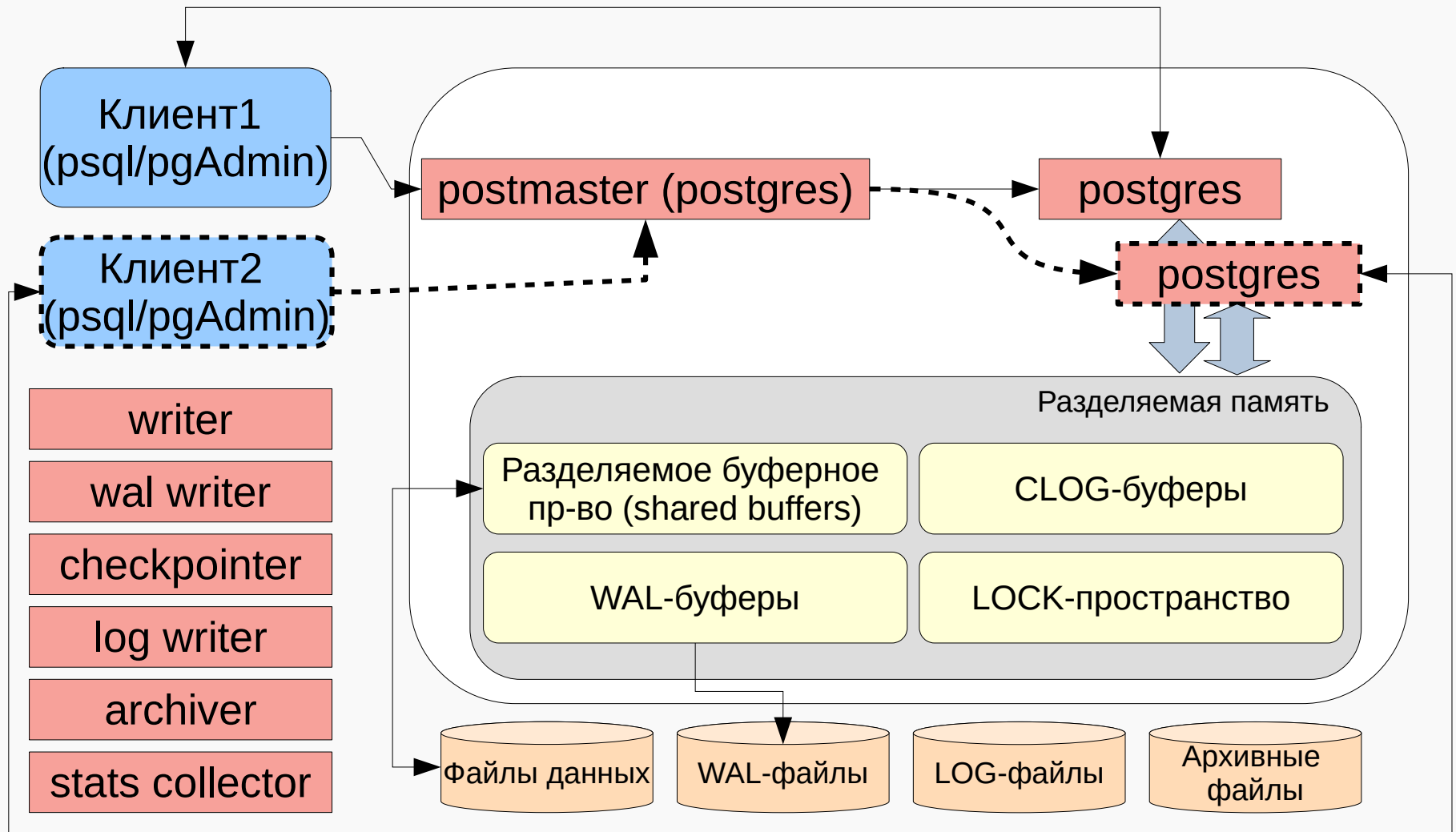
Структуру СУБД можно рассматривать по-разному:

- На уровне структур в основной памяти ЭВМ.
- На уровне процессов в ОС.
- На уровне структуры хранилища данных в ФС.

Кластер БД — множество баз данных, управляемых экземпляром сервера БД (PostgreSQL).

Инстанс (экземпляр) — процессы СУБД + разделяемая память.

Архитектура PostgreSQL



Разделяемое буферное пространство (Shared buffers)

- Хранит копии блоков данных (страниц), считанных из файлов данных.
- Служит для минимизации числа операций обмена с диском.
- Если нужного блока (страницы) данных нет в SB, он читается с диска и помещается в SB.
- Совместно используется всеми фоновыми и пользовательскими процессами экземпляра.
- Можно настроить через `shared_buffers`.
- По умолчанию 128 MB.

Durability

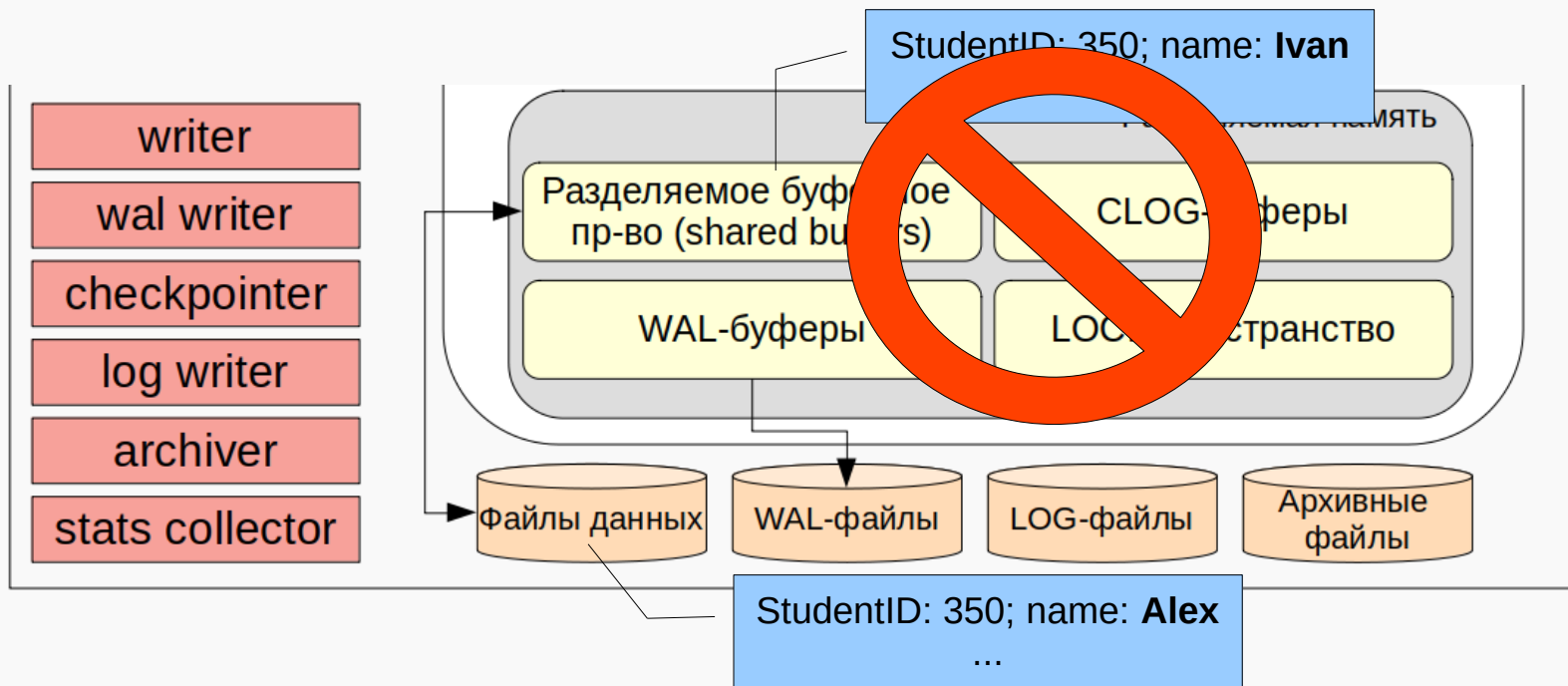
- Долговечность - при успешном завершении транзакции результаты ее работы должны остаться в системе независимо от возможных сбоев оборудования, системы и тд.
- При этом есть shared buffers — в ОП могут быть измененные данные («грязные» страницы), которые не обновлены в файлах данных.
- Синхронизировать файлы данных при каждом изменении неэффективно.
- Что если сбой произошел до того момента, как измененное содержимое SB попало в файлы данных?



Durability

UPDATE STUDENT

```
SET Name = 'Ivan' WHERE studentID = 350;
```



WAL buffers

- WAL — Write Ahead Log
- Хранит информацию об изменениях данных в БД — записи XLOG.
- Изменениям присваивается LSN — log sequence number.
- Эта информация используется для воссоздания актуального состояния данных в случае восстановления базы данных (например, после сбоя).
- Настраивается через параметр `wal_buffers`.

CLOG buffers

- CLOG — Commit Log.
- Хранит данные о статусе проведения транзакций (в процессе, закончена, ...).
- Размер автоматически устанавливается СУБД.
- Доступен серверным процессам.

Пространство для хранения данных о блокировках (Lock space)

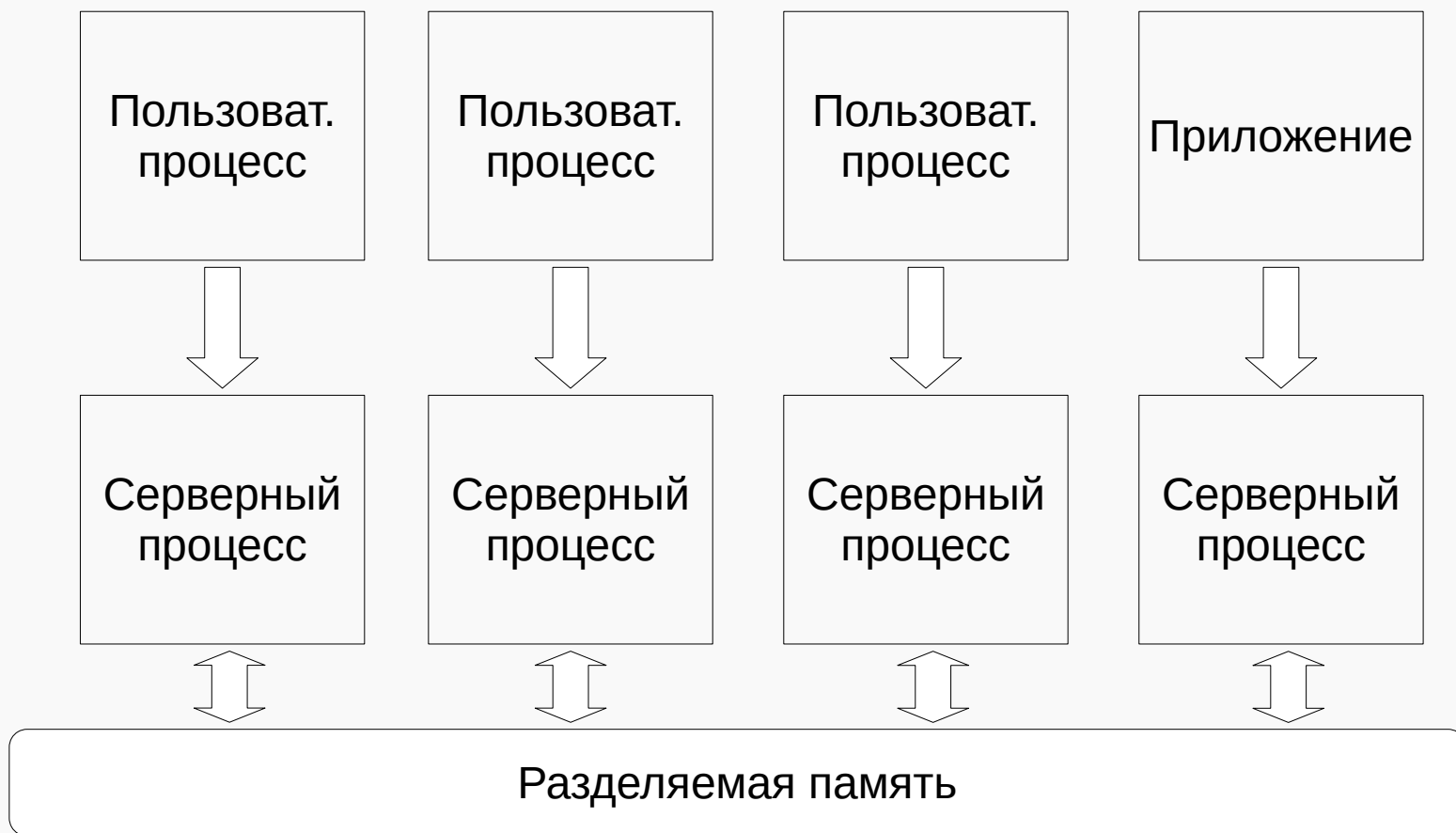
- Хранит данные о блокировках, использующихся экземпляром БД.
- Данные о блокировках доступны всем серверным процессам.
- Можно настроить через `max_locks_per_transaction`.

Буферное пространство процессов БД (неразделяемая область памяти процессов)

- Для каждого серверного пользовательского процесса выделено пространство для осуществления операций.
- По умолчанию — 4 MB.
- Может быть различных видов:
 - vacuum buffers;
 - рабочая память (work_mem) - DISTINCT, ORDER BY, JOIN;
 - вспомогательная рабочая память (maintenance_work_mem) — REINDEX;
 - temp_buffer — для работы со временными таблицами.

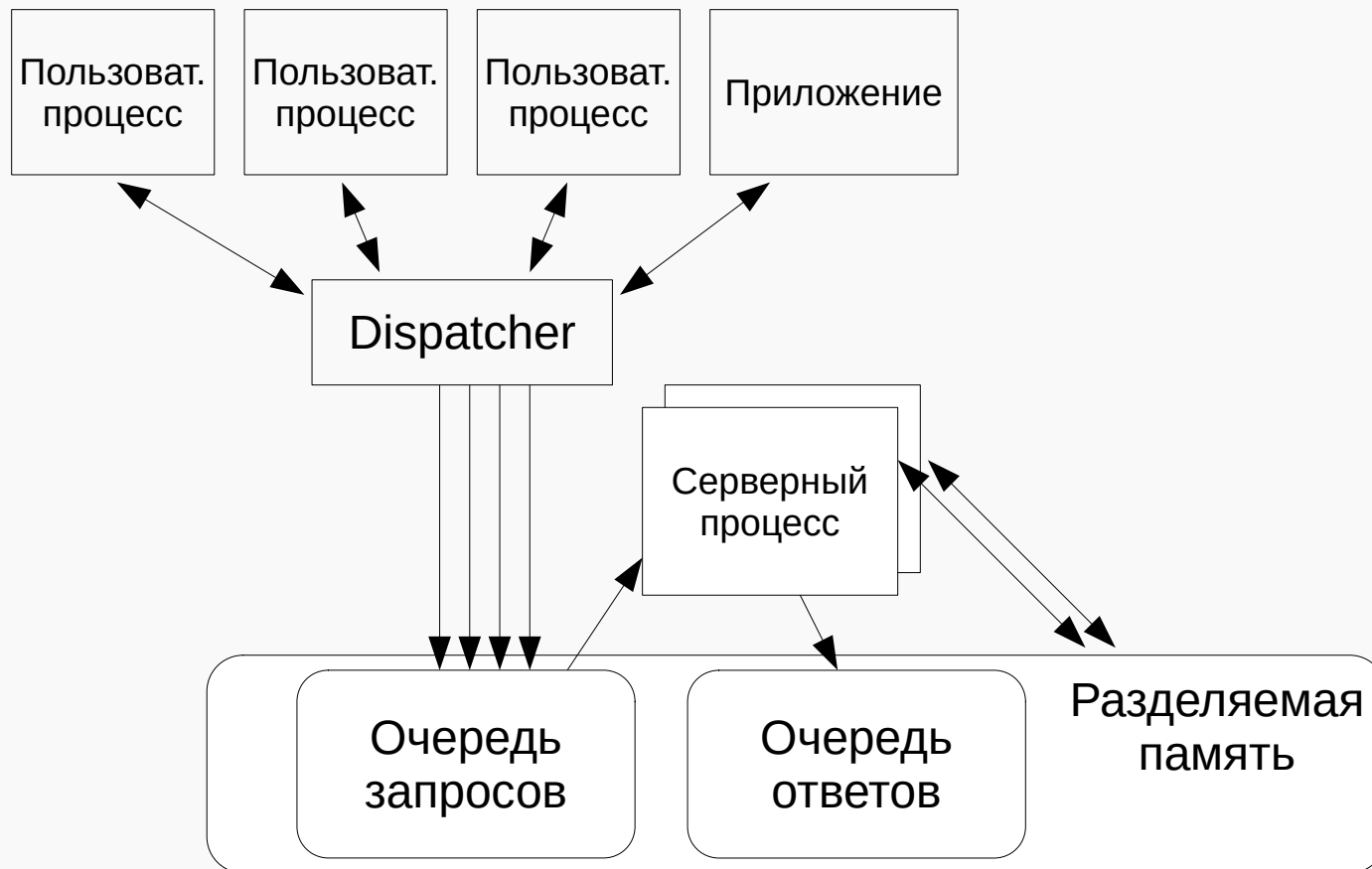
Выделенный сервер (Dedicated Server)

В режиме выделенного сервера каждому пользовательскому процессу создаётся свой «персональный» серверный.



Разделяемый сервер (Dedicated Server)

В режиме разделяемого сервера для каждого пользовательского процесса серверный процесс выделяются диспетчером из специального пула.

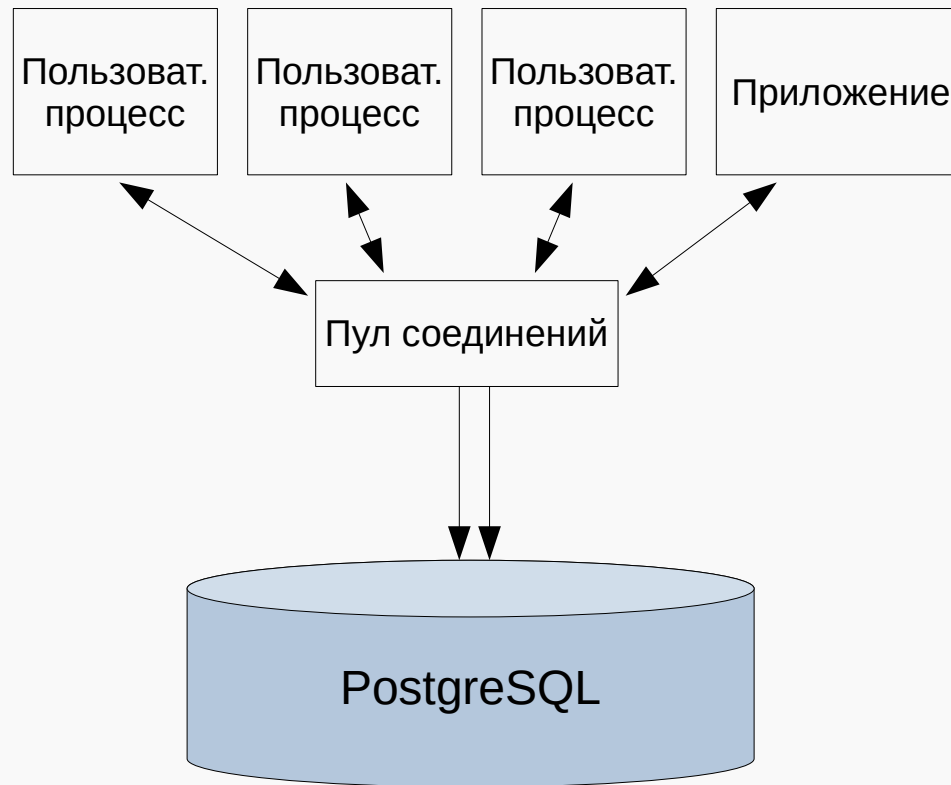


postmaster (postgres)

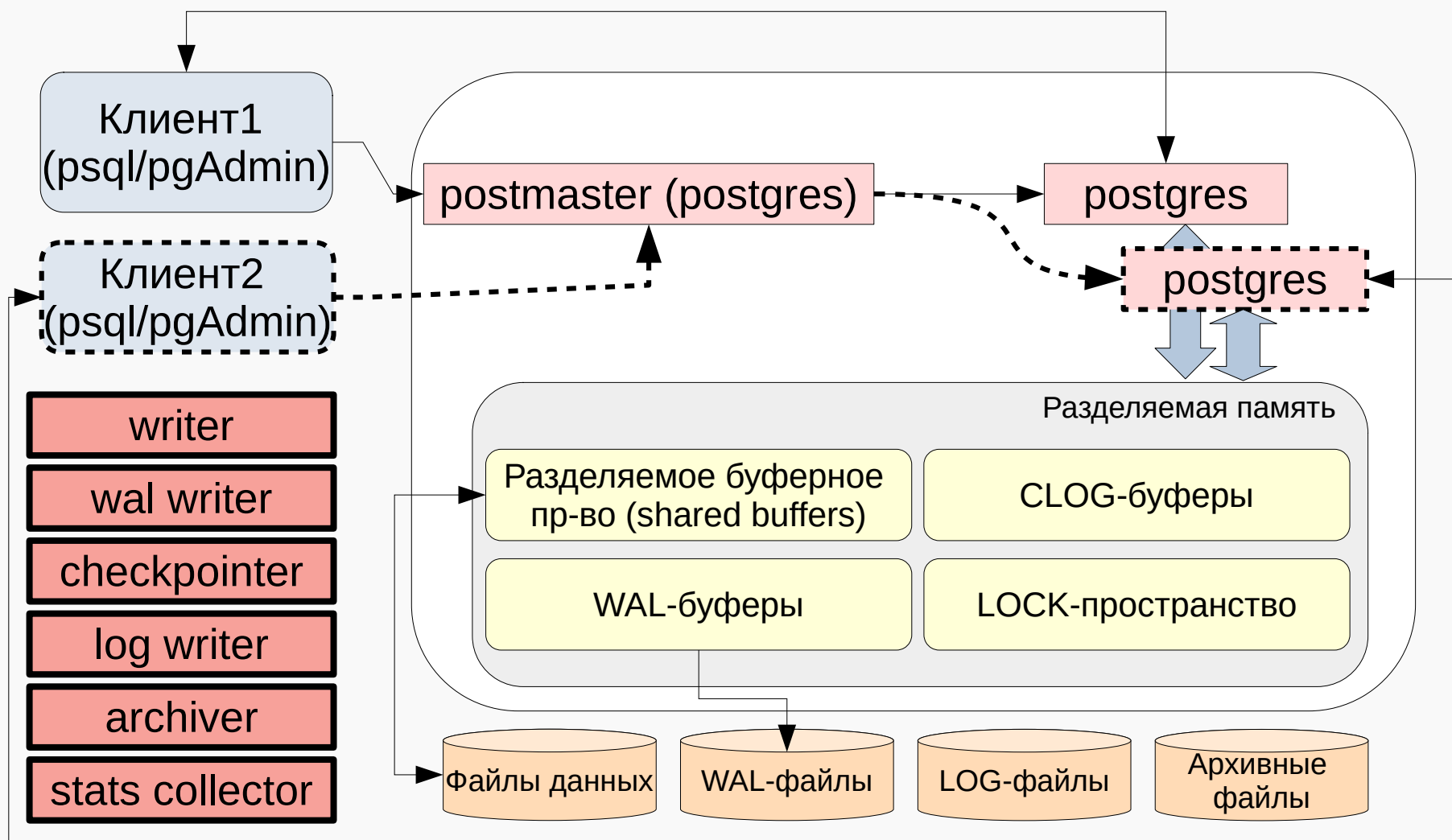
- Представляет сервер PostgreSQL.
- Отвечает за запуск и остановку PostgreSQL.
- Создает серверные процессы для обработки соединений.
- Управляет данными одного кластера БД.
- Кластер БД — БД или несколько БД, расположенных по определенному адресу в файловой системе.

Connection Pool

- pgBouncer
- pgpool-II



Фоновые процессы PostgreSQL



Процесс записи в БД writer process (background writer)

- Периодически записывает измененные (заполненные, «грязные») страницы из SB на диск в файлы данных.
- Помечает записанные страницы «чистыми».
- «Облегчает» работу процесса checkpointer.
- bgwriter_delay

Работа с контрольными точками checkpoint

- *Контрольная точка (checkpoint)* — точки в последовательности транзакций, в которые произведена синхронизация результатов выполненных операций с файлами на диске.
- Создание контрольной точки:
 - ✓ WAL-буфер синхронизируется с диском.
 - ✓ «грязные страницы» записываются на диск.
 - ✓ Контрольная точка фиксируется в логах.
- Контрольная точка создается, когда заполнен `max_wal_size` или через время `checkpoint_timeout` (по умолчанию — 300 секунд) — в зависимости от того, что будет раньше.

logging collector, archiver

logging collector:

- Записывает сообщения, отправленные в stderr, в лог-файлы.
- Для работы нужно, чтобы был установлен параметр `logging_collector`.

archiver:

- Копирует созданные WAL-файлы в указанное место.
- По умолчанию выключен.

stats collector

- Служит для сбора различных статистических данных о БД.
- Статистика хранится в промежуточных файлах и через них используется другими процессами.
- Директория с временными файлами определяется параметром `stats_temp_directory`
- `pg_stat_*`:
`pg_stat_database, ...`

2. Системный каталог

Системный каталог

- В PostgreSQL есть возможность получения данных о хранимых данных — метаданных:
 - Когда была создана таблица?
 - Сколько в ней атрибутов и какого они типа?
 - Какие индексы связаны с данной таблицей?
- Для доступа к метаданным используются таблицы и представления — **системные каталоги**;
- У каждой БД — есть схема ***pg_catalog***, в ней — каталоги, относящиеся к этой БД.

- Использование системных каталогов напрямую (SELECT * FROM pg_*):
SELECT count(*) FROM pg_stat_activity;
- Использование INFORMATION_SCHEMA.
- Мета-команды psql.

- **pg_database** — информация о базах данных в кластере БД; создается один для кластера:
- `SELECT datdba FROM pg_database WHERE datname = 'MYDB';`
- **pg_class** — информация о таблицах, представлениях, индексах и тд.
- **pg_tables** — информация о таблицах текущей БД — у каждого БД свой:
`SELECT * FROM pg_catalog.pg_tables;`
- Функции: `current_user`, `current_schema`, ...

INFORMATION_SCHEMA

- Представления, через которые можно получить данные из системных каталогов.
- Являются частью SQL-стандарта.
- Доступны в PostgreSQL, начиная с версии 8.0.

```
SELECT Table_Name  
FROM information_schema.TABLES;
```

Мета-команды psql

- Посмотреть структуру таблицы:
`\d MyTable`
- Информация об объекте базы данных:
`\d objectName`
- Посмотреть созданные индексы:
`\di`
- ...

oid - идентификатор объекта в PostgreSQL

pg_attribute - каталог с информацией о колонках таблицы.

Некоторые поля:

- **attrelid** - oid таблицы, к которой относится эта колонка (из каталога **pg_class**);
- **attname** - имя колонки.

Для типа oid существуют различные **типы-алиасы** для упрощения работы (например, **regclass** ~ oid таблицы, позволяет получить значение oid по названию таблицы).

Пример

Если в БД одна таблица STUDENT, название колонок этой таблицы:

```
SELECT attname FROM pg_attribute  
WHERE attrelid = (SELECT oid FROM pg_class  
WHERE relname = 'STUDENT');
```

```
SELECT attname FROM pg_attribute WHERE attrelid =  
'STUDENT'::regclass;
```

Пример работы с pg_tables

```
SELECT * FROM pg_catalog.pg_tables;
```

Будет ли работать такой запрос?

```
SELECT * FROM pg_tables;
```

- Используются для логической группировки объектов в БД.
- У каждой БД в PostgreSQL есть схема public.
- Полное имя в PostgreSQL:

`dbName.schemaName.objectName`

`dbName.schemaName1.objName`
`dbName.schemaName2.objName` } Разные объекты

search_path

- Объекты можно использовать без полного имени — тогда используется `search_path`.
- `search_path` — последовательность схем, которая будет использована для идентификации объекта, когда используется неполное имя.
- Схемы рассматриваются в порядке из `search_path`.
- Первая схема из `search_path` — используется для создания объектов — текущая.

search_path

- pg_temp и pg_catalog автоматически добавляются в search_path перед первой указанной схемой (порядок можно переопределить).
- Изменять search_path нужно осторожно — меняется контекст выполнения запросов (разрешение имен).
- show search_path

3. Управление доступом к БД

Пользовательские права

- Разным категориям пользователей должны предоставляться:
 - разные возможности (в зависимости от их потребностей);
 - для управления различных объектов БД.
- Возможности — обеспечение доступа (или выполнения другой операции) с таблицами, представлениями; создание пользователей.

Привилегии

- Предоставляемые возможности определяются **привилегиями**;
- Привилегии:
 - системные — описывают возможность осуществления операций над БД;
 - для взаимодействия с объектами — операции над различными объектами (контроль над данными в объектах БД);
 - с различными объектами связаны различные привилегии.

Работа с привилегиями

```
GRANT privilegeName1, privilegeName2, ...
```

```
[ON table1]
```

```
TO user1, user2, user3;
```

```
GRANT CREATE ON SCHEMA someSch TO sXXXXXXX;
```

- Привилегии: SELECT, INSERT, UPDATE, DELETE, CREATE, EXECUTE, CONNECT, REFERENCES, ...
- ALL PRIVILEGES — для выдачи всех привилегий (в зависимости от контекста).

Владелец объекта

- **Владелец объекта** – обычно пользователь (роль), создавший объект.
- Обладает некоторыми привилегиями для созданного объекта по умолчанию, например:
 - ALTER
 - DROP

Роли

- Привилегии отражают конкретную возможность.
- Роли — именованные наборы привилегий:
 - позволяют управлять объектами и БД на более «высоком» уровне;
 - могут выступать в качестве пользователей;
- Роль - конфигурируется на уровне кластера:
 - назначение и привилегии могут отличаться для разных БД;
 - Имя роли — уникально для кластера.

- **CREATE ROLE:**
 - CREATE ROLE STUDENT;
 - CREATE ROLE STUDENT **WITH LOGIN** PASSWORD 'somePwd';
 - CREATE ROLE STUDADMIN **CREATEROLE**;
- **ALTER ROLE**
- **DROP ROLE**
- Могут использовать суперпользователи, а также роли с CREATEROLE (на не суперпользователях).

- При установке кластера — создается администратор (postgres).
- Созданы для отображения именованных пользователей системы.
- Могут быть созданы суперпользователем и пользователем с ролью CREATEROLE.
- **CREATE USER:**
 - `CREATE USER s234XXX WITH PASSWORD 'somePwd';`

Пользователи

- Создание пользователя аналогично созданию роли с параметром LOGIN:

```
CREATE USER s234XXX;
```

```
CREATE ROLE s234XXX WITH LOGIN;
```
- При создании БД создается роль public;
 - public назначается всем пользователям и ролям — определяет права пользователей и ролей по умолчанию к разным объектам.
- pg_roles

Настройка ролей

- **SUPERUSER** — пользователь может действовать как суперпользователь кластера (все права на все объекты);
 - можно создать нескольких суперпользователей
- **NOSUPERUSER** — убирает возможности SUPERUSER.
- **CREATEROLE** — позволяет создавать роли.
- **CREATEDB** — позволяет создавать базы в кластере.

Настройка ролей (2)

- **PASSWORD** - установить пароль:
 - `CREATE USER s234XXX WITH PASSWORD 'somePwd';`
- **PASSWORD NULL** - запретить пользователю вход по паролю.
- **CONNECTION LIMIT n** — задать ограничение по числу подключений для пользователя.
- **VALID UNTIL** — установить срок действия роли (срок действия пароля этой роли).

Подключение к БД

Для подключения к БД роль должна быть:

- LOGIN;
- содержать привилегию CONNECT на нужную БД;
- разрешение в `pg_hba.conf`;

Группы ролей

- Роли можно объединять в группы для более гибкого управления ими:

```
CREATE ROLE s123456 WITH LOGIN PASSWORD  
'somsdfslid';
```

```
CREATE ROLE STUDENTS;
```

```
GRANT STUDENTS TO s123456;
```

```
CREATE ROLE s123457 WITH LOGIN PASSWORD  
'xfsewrew' IN ROLE students;
```

Админ группы ролей

- Участник группы (или несколько участников группы) могут быть администраторами:

```
CREATE ROLE students WITH NOLOGIN ADMIN  
s123456;
```

- Или с помощью GRANT:

```
GRANT students TO s123456 WITH ADMIN OPTION;
```

- Роль-администратор может добавлять новых участников в группу (как и суперпользователь кластера).
- Роль должна быть создана, чтобы быть добавлена в качестве администратора.

Работа с ролями

- WITH GRANT OPTION — указывается возможность дальнейшей передачи роли от того, кому она назначена:

GRANT UPDATE ON STUDENT TO s458455 WITH GRANT OPTION;
- INHERIT/NOINHERIT — наследование привилегий при работе с группами.
- SET ROLE
- Удаление группы не удаляет ее участников.

INHERIT

```
CREATE ROLE s123456 WITH LOGIN PASSWORD  
'somsdfsl';
```

Создали пользователя
для студента

```
REVOKE ALL ON stud_comments FROM s123456;
```

Убираем все права
на таблицу stud_comments

```
INSERT INTO stud_comments VALUES ...;
```

Ошибка: нет нужных прав

INHERIT

```
CREATE ROLE STUDENTS WITH NOLOGIN;
```

Создали группу ролей
для студентов

```
GRANT INSERT ON stud_comments TO STUDENTS;
```

Выдали права студентам
на изменение таблицы

INHERIT

```
GRANT STUDENTS TO s123456;
```

Добавляем s123456
в группу

```
psql -U s123456 ucheb
```

```
INSERT INTO stud_comments VALUES ...;
```

Значения добавлены

INHERIT

- Участники группы получают права ролей-групп их окружающих.
- INHERIT по умолчанию.

```
CREATE ROLE s123456 WITH LOGIN PASSWORD  
'somsdfsl';
```

```
CREATE ROLE s123456 WITH LOGIN PASSWORD  
'somsdfsl' INHERIT;
```

NOINHERIT

```
CREATE ROLE s123457 WITH LOGIN PASSWORD  
'somsdfslld' NOINHERIT;  
GRANT INSERT ON stud_comments TO STUDENTS;  
GRANT STUDENTS TO s123457;
```

```
psql -U s123457 ucheb
```

```
INSERT INTO stud_comments VALUES ...;
```

Ошибка: нет нужных прав

SET ROLE

```
psql -U s123457 ucheb
```

```
SET ROLE TO STUDENTS;
```

```
INSERT INTO stud_comments VALUES ...;
```

Значения добавлены

Поиск привилегий

- 1) Поиск среди привилегий роли.
- 2) Поиск среди родителей (если у изначальной роли INHERIT). Поиск среди прародителей (если у родителей INHERIT).
- 3) Есть ли привилегия для роли public.

Отмена привилегий

```
REVOKE privilegeName1, privilegeName2, ...
```

```
[ON table1]
```

```
TO user1, user2, user3;
```

```
REVOKE UPDATE, DELETE, TRUNCATE ON STUDENT  
FROM s4343453;
```

Документация PostgreSQL:

<https://www.postgresql.org/docs/14/index.html>

Лицензия PostgreSQL:

PostgreSQL is released under the PostgreSQL License, a liberal Open Source license, similar to the BSD or MIT licenses.

PostgreSQL Database Management System
(formerly known as Postgres, then as Postgres95)

Portions Copyright © 1996-2022, The PostgreSQL Global Development Group

Portions Copyright © 1994, The Regents of the University of California

Permission to use, copy, modify, and distribute this software and its documentation for any purpose, without fee, and without a written agreement is hereby granted, provided that the above copyright notice and this paragraph and the following two paragraphs appear in all copies.

IN NO EVENT SHALL THE UNIVERSITY OF CALIFORNIA BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, INCLUDING LOST PROFITS, ARISING OUT OF THE USE OF THIS SOFTWARE AND ITS DOCUMENTATION, EVEN IF THE UNIVERSITY OF CALIFORNIA HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE UNIVERSITY OF CALIFORNIA SPECIFICALLY DISCLAIMS ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE SOFTWARE PROVIDED HEREUNDER IS ON AN "AS IS" BASIS, AND THE UNIVERSITY OF CALIFORNIA HAS NO OBLIGATIONS TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

Лекция 3

Лектор:
Николаев В.В.

1. Установка и запуск PostgreSQL

Установка PostgreSQL

2 варианта:

- Графический установщик.
- Использование утилит:
 - использование подготовленных бинарных файлов
 - разные сборки для разных ОС;
 - компиляция исходных кодов (make)
 - нет сборки для используемой ОС;
 - нужна самая последняя версия\версия с внесенными изменениями;

Установка PostgreSQL

Базовые компоненты PostgreSQL

- сервер (postgresql);
- клиент (postgresql-client);
- contrib, docs и другие компоненты.

Процесс установки

1. Установка (сборка) базовых компонентов (пакетов):

```
sudo apt install postgresql-XX postgresql-client-XX ...
```

2. Создание системного пользователя (postgres):

```
adduser postgres
```

```
mkdir [PGDATA]
```

```
chown postgres [PGDATA]
```

3. Задание переменных окружения: PGDATA, ...

Процесс установки

4. `[/pg_path]/bin/initdb [-D PGDATA_path]`
5. Запуск экземпляра кластера БД:
 - i. `[/pg_path]/bin/postgres [-D PGDATA_path]`
 - ii. `[/pg_path]/bin/pg_ctl [-D PGDATA_path] -l logfile start`
 - iii. Сервис: `sudo service postgresql start`

Инициализация кластера - initdb

- Создает структуру директорий PGDATA.
- Создает стандартные БД, которые можно использовать для создания своих баз.
- Задаёт локаль и кодировку, которая будет использоваться кластером БД.

```
initdb [-D PGDATA_path]
```

```
pg_ctl initdb
```

Инициализация

Базы данных, доступные после установки:

- пользовательская БД (**postgres**) - принадлежит администратору БД - пользователю postgres;
- создается **template1**;
- создается клон template1 — **template0**;

Создается суперпользователь — по умолчанию совпадает с именем пользователя, который запустил initdb;

- можно задать имя через -U: initdb -U имя

template1, template0

- Изначально template1 и template0 — идентичны.
- **template0** — служит в виде резервной копии.
- **template1** — используется в качестве шаблона при создании других БД.
- Можно создать свою БД, которая будет использоваться в качестве шаблона.
- Если внести изменения в template1 — они будут применены и для баз, созданных на его основе.
- К template0 — нельзя подключиться (она резервная).

Запуск сервера базы данных

- Запуск сервера баз данных:
`[/pg_path]/bin/postgres [-D PGDATA_path]`
- Можно заранее установить переменную PGDATA:
`PGDATA = ...`
`export PGDATA`
`postgres`
- PGDATA - должна быть проинициализирована перед запуском PostgreSQL

Утилита pg_ctl

- Утилита pg_ctl — для упрощения взаимодействия с postgres.
- Можно запускать (по умолчанию в фоне), останавливать, перезапускать кластер:

```
pg_ctl start -l logfile
```

```
pg_ctl status [-D PGDATA_path]
```

- Можно использовать для инициализации кластера:

```
pg_ctl [-D PGDATA_path] initdb
```

2. Подключение к БД PostgreSQL

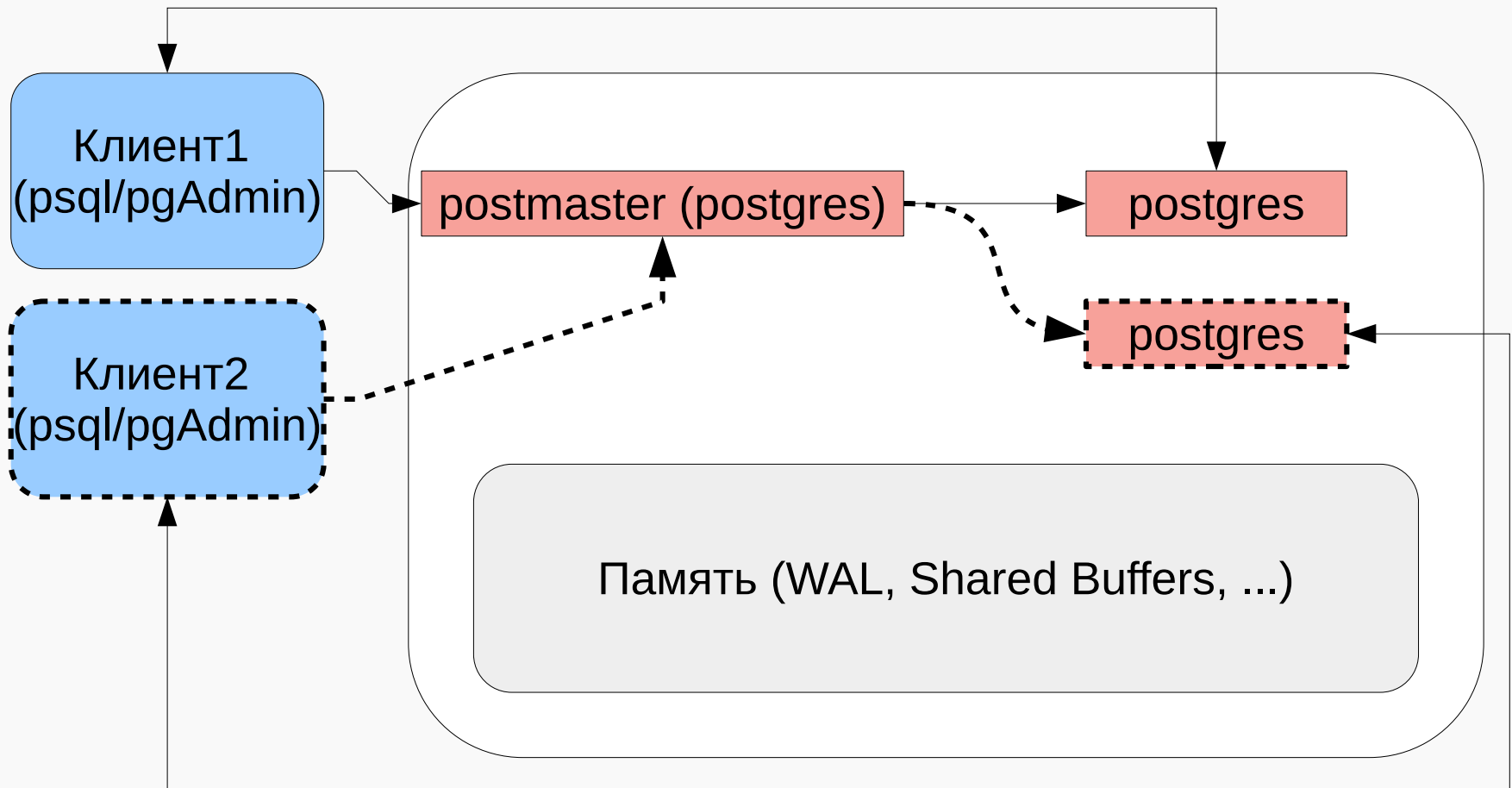
- psql:
 - интерактивный терминал PostgreSQL;
 - стандартный клиент для работы с PostgreSQL;
- pgAdmin:
 - графический клиент;
 - доступен для Windows, Unix, macOS;
- Клиенты, использующие libpq.

Подключение роли к БД

Для подключения к БД роль должна быть:

- LOGIN;
- содержать привилегию CONNECT на нужную БД;
- разрешение в `pg_hba.conf`;

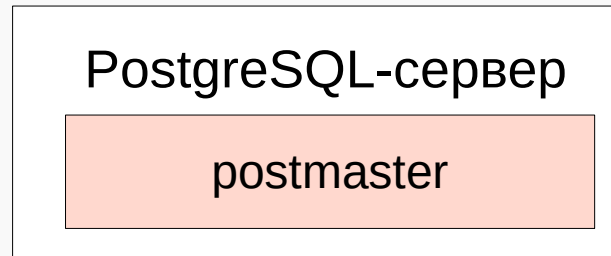
Подключение к PostgreSQL



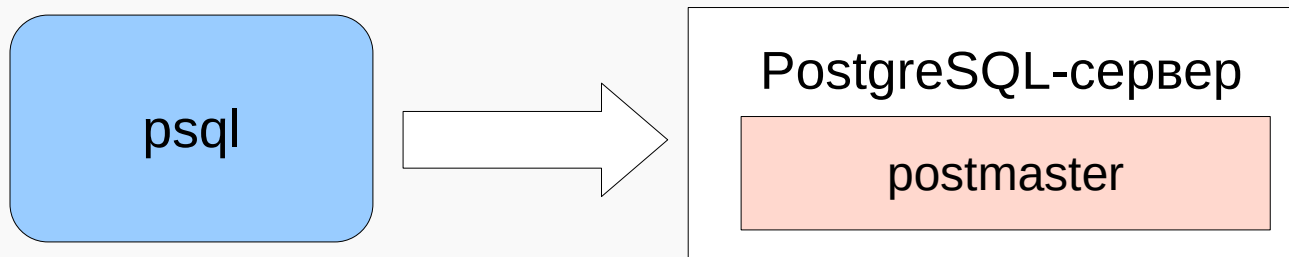
Подключение к серверу БД (1)

1) Запуск сервера PostgreSQL:

```
postgres -p 2378 -D /home/myuser/pgd/data
```

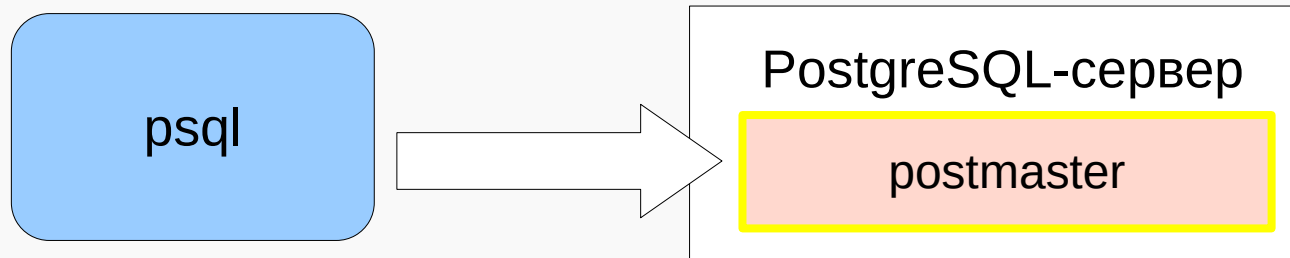


2) Подключаем клиент: `psql -p 2347`

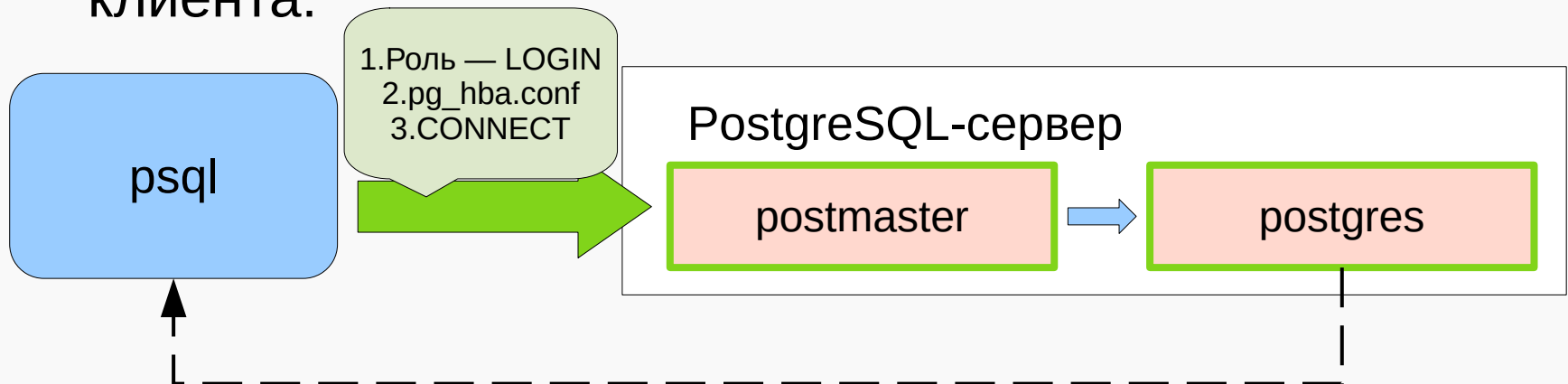


Подключение к серверу БД (2)

3) экземпляр проверяет, может ли клиент получить доступ:



4) Если проверка прошла успешно, создается серверный процесс для обработки соединения данного клиента:



pg_hba.conf

- В pg_hba.conf задаются способы подключения для различных пользователей к различным базам данных.
- Создается при работе initdb.
- По умолчанию располагается в PGDATA.
- Можно поменять через hba_file параметр:
 - при запуске postgres;
 - в postgresql.conf;

Формат pg_hba.conf

- Каждая запись в файле определяет вид подключения для разных категорий пользователей.
- Порядок расположения записей влияет на права:
 - чтение происходит последовательно;
 - pg_hba.conf читается во время запуска;
 - Если файл изменен:
 - ✓ pg_reload_conf()
 - ✓ pg_ctl reload

Пример pg_hba.conf

PostgreSQL Client Authentication Configuration File

...

local DATABASE USER METHOD [OPTIONS]

host DATABASE USER ADDRESS METHOD [OPTIONS]

...

"local" is for Unix domain socket connections only

local all all peer

IPv4 local connections:

host all all 127.0.0.1/32 md5

...

Виды подключений

- Локальный UNIX-сокет (**local**):
 - `psql` (без имени хоста), используется `libpq`:
 - подключается пользователь системы к БД с тем же именем, а также есть пользователь БД с тем же именем.
 - клиент и сервер на одной машине;
 - параметр `unix_socket_directories`
- TCP/IP соединение (**host**):
 - указывается хост, порт:
`psql -h host -p port database`
`psql -h host -p port -U username database`
 - Предполагается, что задан параметр `PGDATA`.

Методы подключений

- **trust** — предоставить доступ всем из данной категории (**опасно!**);
- **по паролю** (scram-sha-256, md5, password). Смена пароля:
 - изменение роли;
 - \password
- **Ident** — похоже на peer, но для tcp/ip (host).
- **peer** — сравнивается пользователь ОС с пользователем БД:
 - можно задать правила отображения пользователей в **pg_ident.conf** и параметр **map** в **pg_hba.conf**;
 - только local;
- ...

3. Создание БД в PostgreSQL

Создание Базы Данных

- *CREATE DATABASE*:
CREATE DATABASE newDb1 [WITH опции];
- *createdb* — утилита в директории [/postgresql]/bin:
createdb -h host -p port -U user [опции] newDb2;
- Чтобы создать базу пользователь должен быть:
 - суперпользователем;
 - обладать ролью *CREATEDB*;

Создание базы из указанного шаблона (1)

- По умолчанию при создании новой базы данных копируется **template1**.
- Можно использовать в качестве шаблона другую базу:
`CREATE DATABASE newDb3 TEMPLATE newDB1;`
`createdb -T newDB2 newDB4`

Создание базы из указанного шаблона (2)

- Чтобы база могла быть шаблоном нужно, чтобы к ней не было подключений **других пользователей**.
- Системный каталог **pg_database**:
 - datistemplate** — БД создана, чтобы быть шаблоном;
 - может быть использована (как шаблон)
владельцем, суперпользователями и пользователями с CREATEDB;
 - dataallowconn** — запрещаются новые подключения.

4. Файловая структура и конфигурация PostgreSQL

Структура файлов PostgreSQL

- **PGDATA** – путь к директории данных кластера, установлена заранее.
 - **global** – содержит таблицы уровня кластера (pg_database).
 - **pg_wal** – директория с WAL-файлами.
 - **pg_xact** – директория с данными о коммитах транзакций (CLOG).
 - **base** – файлы данных базы данных кластера:
 - 1
 - 13201
 - 13202

Просмотр файлов данных (1)

```
sudo ls /PGDATA/base
```

В результате получим список oid, соответствующих базам:

- 1
- 13201
- 13202

Просмотр файлов данных (2)

Утилита `oid2name` — для просмотра имен (по `oid` или `filenode`):

`oid2name`

Oid	Database Name	Tablespace
13202	template1	pg_default
13201	template0	pg_default
1	postgres	pg_default

Файлы данных

- Объектам БД (PostgreSQL) — соответствуют файлы данных:
 - если до 1Gb (по умолчанию): объекту соответствует 1 файл данных;
 - если $> 1\text{Gb}$: объекту соответствуют файлы-сегменты;
- Находятся в `$PGDATA/base` в директории соответствующей БД.

Базовые конфигурационные файлы

- Стандартный путь, по которому располагаются конфигурационные файлы:
`/etc/postgresql/версия/main/`
- **postgresql.conf** — базовый файл для хранения настроек.
- **postgresql.auto.conf** — динамически изменяемые настройки (через ALTER SYSTEM).
- **pg_hba.conf** — конфигурация подключений к БД.
- **pg_ident.conf** — файл отображения имен.

postgresql.conf

- **postgresql.conf** — базовый файл для хранения настроек.
- Создается при работе initdb.
- По умолчанию располагается в PGDATA.
- Представляет из себя набор параметров (имя, значение):
 `search_path = 'someSchema, public'`
 `shared_buffers = 128MB`
- Определяют значения для всего кластера.

Изменение конфигурационных параметров

1. `postgresql.conf/ALTER SYSTEM` — на уровне кластера;
2. `ALTER DATABASE` — изменить 1. на уровне базы данных;
3. `ALTER ROLE` — переписать 1. и 2. на уровне пользователя.
4. `SET` — изменить для сессии:
`SET param TO value/DEFAULT;`
5. Параметры можно задать в команде запуска сервера БД (`postgres`) — для переписи параметров `postgresql.conf`.

Значения применяются
при запуске новой сессии

5. Табличные пространства

Табличные пространства

- **Табличные пространства** — позволяют задать пути (директорию в файловой системе), где будут храниться объекты БД (вне PGDATA):
 - позволяют управлять физическим расположением объектов БД.
- У табличных пространств есть имя.
- Просмотр размера табличного пространства:
 - Функция `pg_tablespace_size('имя_тп')`
- Системный каталог: `pg_tablespace`
- `$PGDATA/pg_tblspc` содержит символические ссылки на директории внешних табличных пространств.

Табличные пространства (1)

- Табличные пространства можно задать для разных объектов БД: таблиц, индексов, последовательностей, представлений.

```
CREATE TABLESPACE newTablespace LOCATION  
'/diskA/someDir';
```

```
CREATE TABLE STUDENT (  
    id integer NOT NULL  
) TABLESPACE newTablespace;
```

Стандартные табличные пространства

- **pg_default** — соответствует директории base в PGDATA;
 - по умолчанию здесь хранятся файлы данных, соответствующие объектам БД.
- **pg_global** — соответствует директории global в PGDATA;
 - pg_database, pg_authid, pg_tablespace, некоторые другие каталоги и индексы.

Табличные пространства (2)

- Табличное пространство можно задать для БД целиком:

```
CREATE DATABASE TestDB  
TABLESPACE newTablespace;
```

- Табличное пространство может быть изменено:

```
ALTER TABLE STUDENT  
SET TABLESPACE pg_default;
```

Зачем нужны табличные пространства?

- Можно контролировать что где хранится.
- Критически важные объекты можно размещать на более быстрых физических дисках.
- Если заканчивается место на жестком диске, можно использовать другой диск для размещения объектов.

Документация PostgreSQL:

<https://www.postgresql.org/docs/14/index.html>

Лицензия PostgreSQL:

PostgreSQL is released under the PostgreSQL License, a liberal Open Source license, similar to the BSD or MIT licenses.

PostgreSQL Database Management System
(formerly known as Postgres, then as Postgres95)

Portions Copyright © 1996-2022, The PostgreSQL Global Development Group

Portions Copyright © 1994, The Regents of the University of California

Permission to use, copy, modify, and distribute this software and its documentation for any purpose, without fee, and without a written agreement is hereby granted, provided that the above copyright notice and this paragraph and the following two paragraphs appear in all copies.

IN NO EVENT SHALL THE UNIVERSITY OF CALIFORNIA BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, INCLUDING LOST PROFITS, ARISING OUT OF THE USE OF THIS SOFTWARE AND ITS DOCUMENTATION, EVEN IF THE UNIVERSITY OF CALIFORNIA HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE UNIVERSITY OF CALIFORNIA SPECIFICALLY DISCLAIMS ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE SOFTWARE PROVIDED HEREUNDER IS ON AN "AS IS" BASIS, AND THE UNIVERSITY OF CALIFORNIA HAS NO OBLIGATIONS TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

Лекция 4

Лектор:
Николаев В.В.

1. Транзакции

Транзакция

- Транзакции объединяют последовательность действий в одну операцию.
- Промежуточные состояния внутри последовательности операций **не видны** другим транзакциям.
- Если что-то помешает успешно завершить транзакцию, ни один из результатов этих действий не сохранится в базе данных.

ACID

- Atomicity (Атомарность)
- Consistency (Согласованность)
- Isolation (Изолированность)
- Durability (Устойчивость, долговечность)

Atomicity (атомарность):

- гарантирует, что результаты работы транзакции не будут зафиксированы в системе частично;
- будут либо выполнены все операции в транзакции, либо не выполнено ни одной.

Consistency (согласованность):

- после выполнения транзакции база данных должна быть в согласованном (**целостном**) состоянии;
- во время выполнения транзакции (после выполнения отдельных операций в рамках транзакции) согласованность не требуется.

Isolation (изолированность):

- во время выполнения транзакции другие транзакции (выполняющиеся параллельно) не должны оказывать влияние на результат транзакции.

Durability (долговечность):

- при успешном завершении транзакции результаты ее работы должны остаться в системе независимо от возможных сбоев оборудования, системы и тд.

BEGIN;

UPDATE BANK_ACCOUNT

SET AccValue = AccValue+1000 WHERE Client_ID = 1;

UPDATE BANK_ACCOUNT

SET AccValue = AccValue-1000 WHERE Client_ID = 2;

COMMIT;

- ROLLBACK;
- SAVEPOINT;

Какими бывают транзакции?

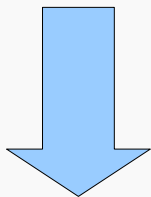
- **Неявные** (implicit) — СУБД начинает без явного указания;
 - любое действие осуществляется в контексте некоторой транзакции.
- **Явные** (explicit) — транзакции, которые задает пользователь самостоятельно (BEGIN, COMMIT).
- Если нет явного указания начала транзакции, любое предложение выполняется в рамках **неявной транзакции**;

Неявные транзакции

- `SELECT * FROM STUDENTS;`
- `INSERT INTO STUDENTS VALUES ...`

Неявные транзакции

```
SELECT * FROM STUDENTS;
```



```
BEGIN;
```

```
SELECT * FROM STUDENTS;
```

```
COMMIT;
```

BEGIN;

CREATE TABLE STUDENTS (StudId int, Name text);

CREATE TABLE EXAMS (ExamId int, ExName text);

ROLLBACK;

2. Время и транзакции

Как узнать время в PostgreSQL?

CURRENT_TIME, CURRENT_TIMESTAMP, now()

- Функция CURRENT_TIME:

```
SELECT CURRENT_TIME as my_time;
```

```
-- 13:30:49. ...
```


Как узнать время в PostgreSQL?

Функция CURRENT_TIME:

```
SELECT CURRENT_TIME as my_time_1;
```

```
-- 13:30:49. ...
```

```
-- [ ... ONE HOUR LATER ... ]
```

```
SELECT CURRENT_TIME as my_time_2;
```

```
-- ???
```

Как узнать время в PostgreSQL?

Функция `CURRENT_TIME`:

```
SELECT CURRENT_TIME as my_time_1;
```

```
-- 13:30:49. ...
```

```
-- [ ... ONE HOUR LATER ... ]
```

```
SELECT CURRENT_TIME as my_time_2;
```

```
-- 14:30:49. ...
```

Как узнать время внутри транзакции?

```
BEGIN;  
  
SELECT CURRENT_TIME as my_time_1;  
  
-- 13:30:49. ...  
  
-- [ ... ONE HOUR LATER ... ]  
  
SELECT CURRENT_TIME as my_time_2;  
  
-- ???  
  
COMMIT;
```

Как узнать время в PostgreSQL?

```
BEGIN;  
  
SELECT CURRENT_TIME as my_time_1;  
  
-- 13:30:49. ...  
  
-- [ ... ONE HOUR LATER ... ]  
  
SELECT CURRENT_TIME as my_time_2;  
  
-- 13:30:49. ...  
  
COMMIT;
```

Время и транзакции

- **CURRENT_TIME** — возвращает время начала транзакции (CURRENT_TIMESTAMP, transaction_timestamp);
- **clock_timestamp()** — возвращает текущее время (timestamp);

```
BEGIN;  
  
SELECT CURRENT_TIME as my_time_1;  
  
-- 13:30:49. ...  
  
-- [ ... ONE HOUR LATER ... ]  
  
SELECT clock_timestamp() as my_time_2;  
  
-- 2022-04-25 14:30:49. ...  
  
COMMIT;
```

3. Идентификация транзакций

Идентификатор транзакции (xid)

- Идентификатор транзакции (xid) — назначается СУБД для каждой новой транзакции;
 - xid — уникален;
- При создании/модификации записи хранится xid транзакции.

pg_current_xact_id()

- Возвращает xid текущей транзакции (bigint):
 - `SELECT pg_current_xact_id();` -- 7435
 - `txid_current()`
- Что можно сказать о выводе `txid_current()`?
 - `SELECT Column1, pg_current_xact_id()`
`FROM TABLE1;` -- **(1) ???**
 - `SELECT Column2, pg_current_xact_id()`
`FROM TABLE2;` -- **(2) ???**

Идентификатор транзакции (xid)

- Идентификатор транзакции (xid) — назначается СУБД для каждой новой транзакции;
 - xid — уникален.
- 32 бита, беззнаковое число.
- При создании/модификации записи хранится xid транзакции.

Как он хранится?

Как можно посмотреть xid?

```
SELECT * FROM STUDENTS;
```

STUDENTS

Stud_ID	Name	Surname	Group
1	Ivan	Ivanov	33313
2	Petr	Petrov	33314

Таблицы — не то, чем кажутся



Stud_ID	Name	Surname	Group
1	Ivan	Ivanov	33313
2	Petr	Petrov	33314

Как можно посмотреть xid?

```
SELECT xmin, * FROM STUDENTS;
```

STUDENTS

xmin	Stud_ID	Name	Surname	Group
345	1	Ivan	Ivanov	33313
345	2	Petr	Petrov	33314

xid транзакции

Скрытые (системные) поля

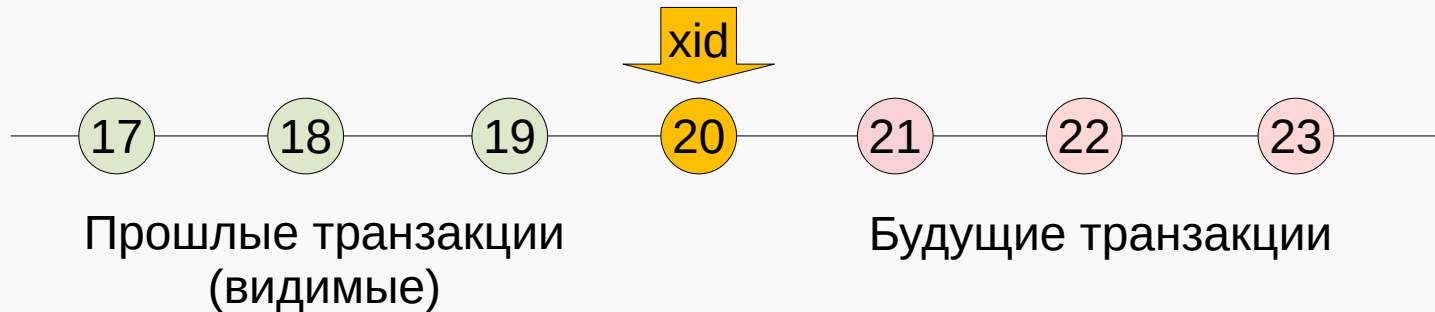
- **xmin** — хранит xid транзакции, в рамках которой запись (копия записи) была создана;
- **xmax** — хранит xid транзакции, в рамках которой запись была удалена;
- **cmin** — идентификатор команды, создавшей запись;
- **cmah** — идентификатор команды, удалившей запись;

xid — возможные значения

- xid — 32 бита, беззнаковое.
- В **простейшем** случае: чем больше xid транзакции — тем позже она началась:
 - xid до: прошлое
 - xid после: будущее
- xid — начинается с 3 (значения 0, 1, 2 — зарезервированы).

Могут ли xid закончиться?

- Начинается выдача значений заново (с 3);



- В простейшем случае: чем больше xid транзакции — тем позже она началась.

Проблема: что делать, когда перешли через MAX: **xid wraparound problem**

xid wraparound problem

- Установка статуса неактуальных записей:
 - Статус **FROZEN** — для неактуальных записей (~ транзакции для которых уже давно завершены).
- Экономия xid:
 - если в транзакции нет операций, изменяющих состояние БД, ей выдается виртуальный xid.
- WARNING: database test must be vacuumed within [...] transactions ...
 - **ПАНИКА!**

Выдача xid для транзакции

BEGIN;

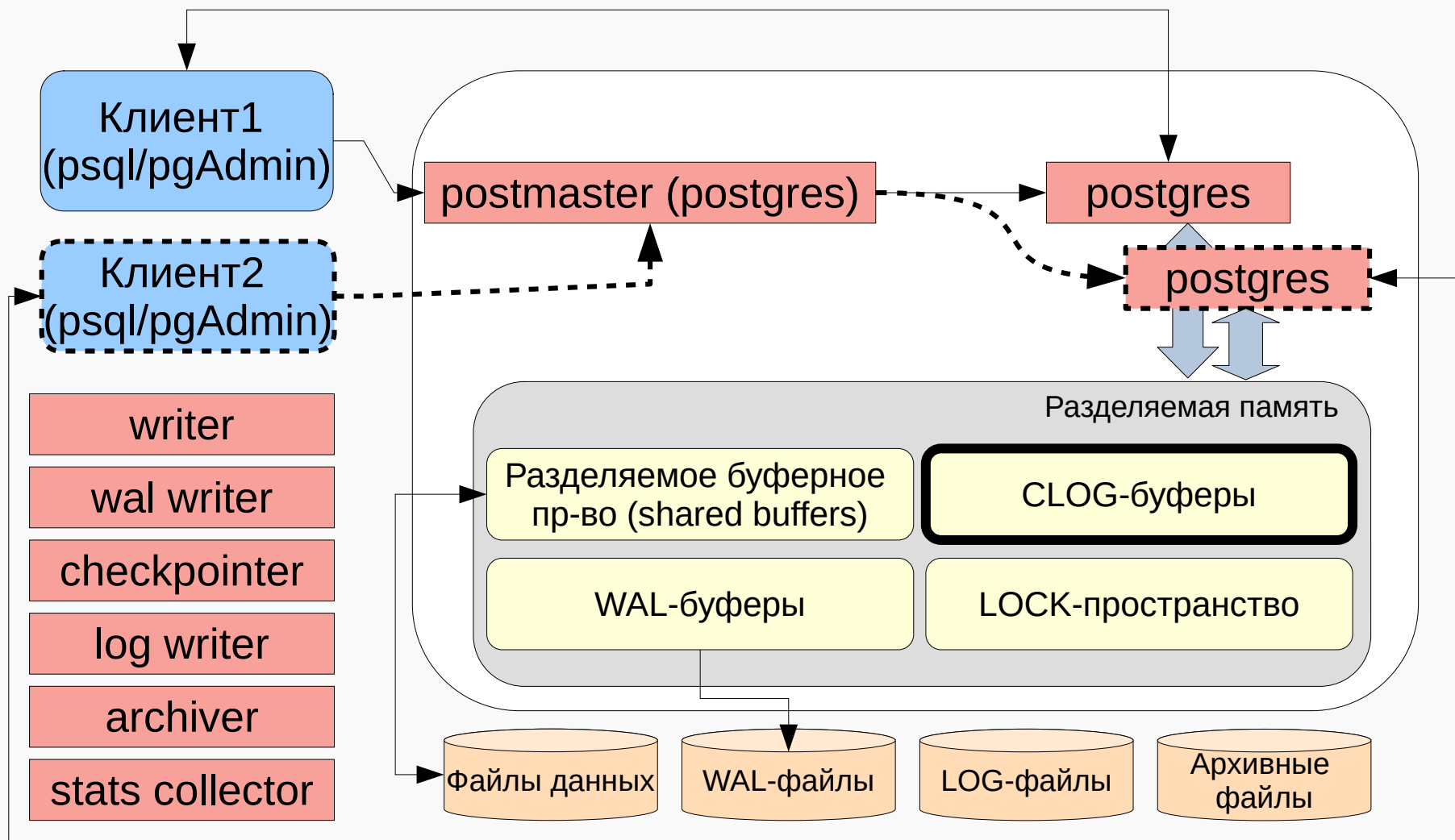
SELECT * FROM STUDENTS;

UPDATE STUDENTS SET Group = 3100 ... ;

COMMIT;

Выдача xid:
SELECT txid_current_if_assigned();
-- xid текущей транзакции
-- (если xid не назначен — NULL)

CLOG-буфер



CLOG-буфер

- Хранится статус транзакций:
 - IN_PROGRESS
 - COMMITTED
 - ABORTED
 - SUB_COMMITTED
- Файлы — в директории pg_xact;

...	...
350	COMMITTED
351	ABORTED
352	ABORTED
353	COMMITTED
...	...

4. Реализация транзакций в PostgreSQL

Реализация транзакций

- Транзакции должны удовлетворять требованиям ACID.
- СУБД должна обеспечивать многопользовательский доступ:
 - разные пользователи могут запрашивать и изменять одни и те же данные в одно время.
- Существуют разные способы реализации транзакций:
 - могут основываться на блокировании ресурсов, синхронизации доступа к данным, создании копий данных, ...

Реализация транзакций в PostgreSQL

- Реализация транзакций (DML) в PostgreSQL — **MVCC**;
- MVCC — Multi-Version Concurrency Control
 - в PostgreSQL: Serializable Snapshot Isolation (SSI).
- Особенности MVCC (SSI):
 - при изменении данных в транзакции — создается новая копия данных;
 - существующие копии данных не изменяются;
 - при выборке данных берется подходящая для данной транзакции версия данных;

MVCC в PostgreSQL. Пример

STUDENTS

Stud_ID	Name	Surname	Group
1	Ivan	Ivanov	33313

```
UPDATE STUDENTS  
SET Group = 33314  
WHERE Stud_ID = 1;
```

STUDENTS

Stud_ID	Name	Surname	Group
1	Ivan	Ivanov	33313
1	Ivan	Ivanov	33314

Добавление данных в таблицу

```
SELECT xmin, * FROM STUDENTS;
```

STUDENTS

xmin	Stud_ID	Name	Surname	Group
345	1	Ivan	Ivanov	33313
345	2	Petr	Petrov	33314

```
BEGIN;
```

```
INSERT INTO STUDENTS VALUES (3, 'Vlad', 'Vladov',  
33313);
```

380

```
SELECT pg_current_xact_id_if_assigned(); COMMIT;
```

```
SELECT xmin, cmin * FROM STUDENTS;
```

xmin	xmin	Stud_ID	Name	Surname	Group
345	0	1	Ivan	Ivanov	33313
345	0	2	Petr	Petrov	33314
380	0	3	Vlad	Vladov	33313

Удаление данных

```
SELECT xmin, * FROM STUDENTS;
```

STUDENTS

xmin	Stud_ID	Name	Surname	Group
345	1	Ivan	Ivanov	33313
345	2	Petr	Petrov	33314
380	3	Vlad	Vladov	33313

```
BEGIN;
```

```
DELETE FROM STUDENTS;
```

390

```
SELECT pg_current_xact_id_if_assigned(); COMMIT;
```

xmin	xmax	cmx	Stud_ID	Name	Surname	Group
345	390	0	1	Ivan	Ivanov	33313
345	390	0	2	Petr	Petrov	33314
380	390	0	3	Vlad	Vladov	33313

Обновление данных

```
SELECT xmin, * FROM STUDENTS;
```

STUDENTS

xmin	Stud_ID	Name	Surname	Group
400	1	Egor	Egorov	33313

```
BEGIN;
```

```
UPDATE STUDENTS SET GROUP=33316;
```

```
SELECT pg_current_xact_id_if_assigned(); COMMIT;
```

410

xmin	xmax	Stud_ID	Name	Surname	Group
400	410	1	Egor	Egorov	33313
410	0	1	Egor	Egorov	33316

«Мертвая» запись

Блокировки данных

- Даже несмотря на использование MVCC в некоторых случаях исполнение транзакции блокируется:

Transaction 1:	Transaction 2:
BEGIN;	
	BEGIN;
UPDATE STUDENTS SET Group = 33314 WHERE Stud_ID = 1 ;	
	UPDATE STUDENTS SET Group = 33315 WHERE Stud_ID = 1 ;
COMMIT;	
	COMMIT;

Transaction 2
блокируется!

Transaction 2
разблокируется!

5. Изоляция транзакций

Виды конфликтов при параллельном доступе к данным

При организации параллельного доступа к данным могут возникнуть различные проблемы:

- Грязное чтение («dirty read»);
- Неповторяющееся чтение («nonrepeatable read»);
- Фантомное чтение («phantom read»);

«Грязное чтение»

Проблема возникает, если одна транзакция видит измененные данные другой (незавершенной транзакции):

Transaction 1:	Transaction 2:				
BEGIN;					
	BEGIN;				
UPDATE STUDENTS SET Group = 33314 WHERE Stud_ID = 1;					
	SELECT * FROM STUDENTS;	STUDENTS			
ROLLBACK;		Stud_ID	Name	Surname	Group
		1	Ivan	Ivanov	33314
	COMMIT;				

Неповторяющееся чтение

Проблема возникает, если один и тот же запрос в рамках транзакции возвращает разные результаты:

Transaction 1:	Transaction 2:				
BEGIN;					
	BEGIN;				
	SELECT * FROM STUDENTS;	STUDENTS			
		Stud_ID	Name	Surname	Group
		1	Ivan	Ivanov	33313
UPDATE STUDENTS SET Group = 33314 WHERE Stud_ID = 1 ;					
COMMIT;		STUDENTS			
	SELECT * FROM STUDENTS;	Stud_ID	Name	Surname	Group
		1	Ivan	Ivanov	33314
	COMMIT;				

Фантомное чтение

Проблема возникает, если один и тот же запрос в рамках транзакции возвращает разное число записей:

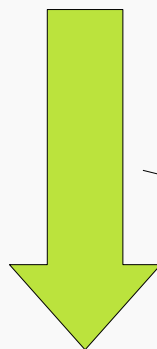
Transaction 1:	Transaction 2:				
BEGIN;					
	BEGIN;				
	SELECT * FROM STUDENTS;	STUDENTS			
		Stud_ID	Name	Surname	Group
		1	Ivan	Ivanov	33313
INSERT INTO STUDENTS VALUES (2, 'Viktor', 'Ivanov', 33315);					
COMMIT;		STUDENTS			
	SELECT * FROM STUDENTS;	Stud_ID	Name	Surname	Group
		1	Ivan	Ivanov	33313
	COMMIT;	2	Viktor	Viktorov	33315

Сами данные не изменяются

Режимы для организации доступа к данным в PostgreSQL

По SQL-стандарту — 4 уровня изоляции:

- 1) Read Uncommitted
- 2) Read Committed
- 3) Repeatable Read
- 4) Serializable



Ограничения строже

Read Uncommitted — не поддерживается PostgreSQL.

Уровни изоляции в PostgreSQL

	Грязное чтение	Неповторяющееся чтение	Фантомное чтение	Аномалии сериализации
Read uncommitted/ нет в PostgreSQL	+	+	+	+
Read committed/ по умолчанию в PostgreSQL	-	+	+	+
Repeatable read	-	-	+(стандарт)/- (PostgreSQL)	+ (write skew/ read-only transaction skew)
Serializable	-	-	-	-

Выбор режима изоляции транзакции

```
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

```
BEGIN;
```

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

Нельзя менять по ходу транзакции!

Snapshot Isolation (SI)

- снимок (snapshot) — характеризует временное окно видимости данных для транзакции:
 - каждая транзакция видит свой набор данных, определяемый связанным с ней снимком;
 - снимок: какие xid (версии данных) видит данная транзакция;
 - можно узнать через `pg_current_snapshot()`.
- `pg_current_snapshot()` (`txid_current_snapshot()`) - возвращает текущий снимок:
 - Формат: **xmin:xmax:xid1,xid2**
 - Верхняя граница (xmax — не включается);

Предоставляется менеджером транзакций:

- READ COMMITTED: свой снимок для каждого выполняемого SQL-предложения.
- REPEATABLE READ, SERIALIZABLE: снимок — при выполнении только первого SQL-предложения.

Выбор подходящей версии записи определяется по **правилам проверки видимости**, которые используют:

- СНИМОК;
- значения $x_{\min}$, $x_{\max}$, $c_{\min}$, $c_{\max}$, ...

READ COMMITTED

Tr 1 (xid = 321):	Tr 2 (xid = 322):				
BEGIN REP. READ;					
	BEGIN READ COM.;				
Snapshot: 321:321	Snapshot: 321:321				
SELECT * FROM STUDENTS;	SELECT * FROM STUDENTS;	Stud_ID	Name	Surname	Group
Snapshot: 321:321		1	Ivan	Ivanov	33313
UPDATE STUDENTS SET Name = 'Egor' WHERE Stud_ID = 1;					
	Snapshot: 321:321				
	SELECT * FROM STUDENTS;	Stud_ID	Name	Surname	Group
		1	Ivan	Ivanov	33313
COMMIT;					
	Snapshot: 322:322				
	SELECT * FROM STUDENTS;	Stud_ID	Name	Surname	Group
		1	Egor	Ivanov	33313
	COMMIT;				

REPEATABLE READ

Tr 1 (xid = 321):	Tr 2 (xid = 322):				
BEGIN REP. READ;					
	BEGIN READ COM.;				
Snapshot: 321:321	Snapshot: 321:321				
SELECT * FROM STUDENTS;	SELECT * FROM STUDENTS;	Stud_ID	Name	Surname	Group
Snapshot: 321:321		1	Ivan	Ivanov	33313
UPDATE STUDENTS SET Name = 'Egor' WHERE Stud_ID = 1;					
	Snapshot: 321:321				
	SELECT * FROM STUDENTS;	Stud_ID	Name	Surname	Group
		1	Ivan	Ivanov	33313
COMMIT;					
	Snapshot: 321:321				
	SELECT * FROM STUDENTS;	Stud_ID	Name	Surname	Group
		1	Ivan	Ivanov	33313
	COMMIT;				

SERIALIZABLE, REPEATABLE READ

Потерянное обновление

Запрещено, чтобы сторонние транзакции изменяли данные, используемые в текущей транзакции:

Tx 1:	Tx 2:
BEGIN ISOLATION LEVEL SERIALIZABLE;	
	BEGIN ISOLATION LEVEL SERIALIZABLE;
UPDATE STUDENTS SET Name = 'Roman' WHERE Stud_ID = 1;	Блокировка Tx 2 До завершения Tx 1
	UPDATE STUDENTS SET Name = Upper(Name) WHERE Stud_ID = 1;
COMMIT;	ERROR: could not serialize access...
Tx 1 завершится успешно	COMMIT;



6. VACUUM

- Реализация транзакций в PostgreSQL — **MVCC**;
- MVCC — Multi-Version Concurrency Control
 - в PostgreSQL: Serializable Snapshot Isolation (SSI).
- Особенности MVCC (SI):
 - при изменении данных в транзакции — **создается новая копия данных**;
 - существующие копии данных **не изменяются**;

С накопленными неактуальными данными нужно что-то делать!

Неактуальные данные

STUDENTS

Stud_ID	Name	Surname	Group
1	Ivan	Ivanov	33313

```
UPDATE STUDENTS  
SET Group = 33314  
WHERE Stud_ID = 1;
```

После выполнения
транзакции эта строка
больше не нужна,
она занимает место

STUDENTS

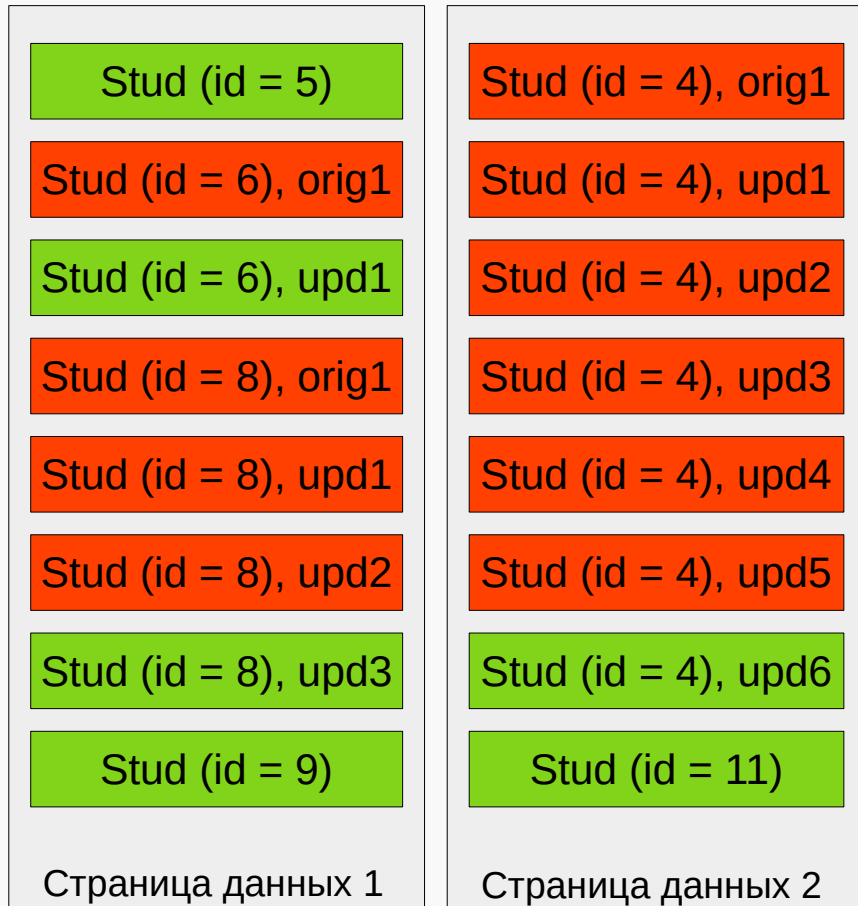
Stud_ID	Name	Surname	Group
1	Ivan	Ivanov	33313
1	Ivan	Ivanov	33314

VACUUM

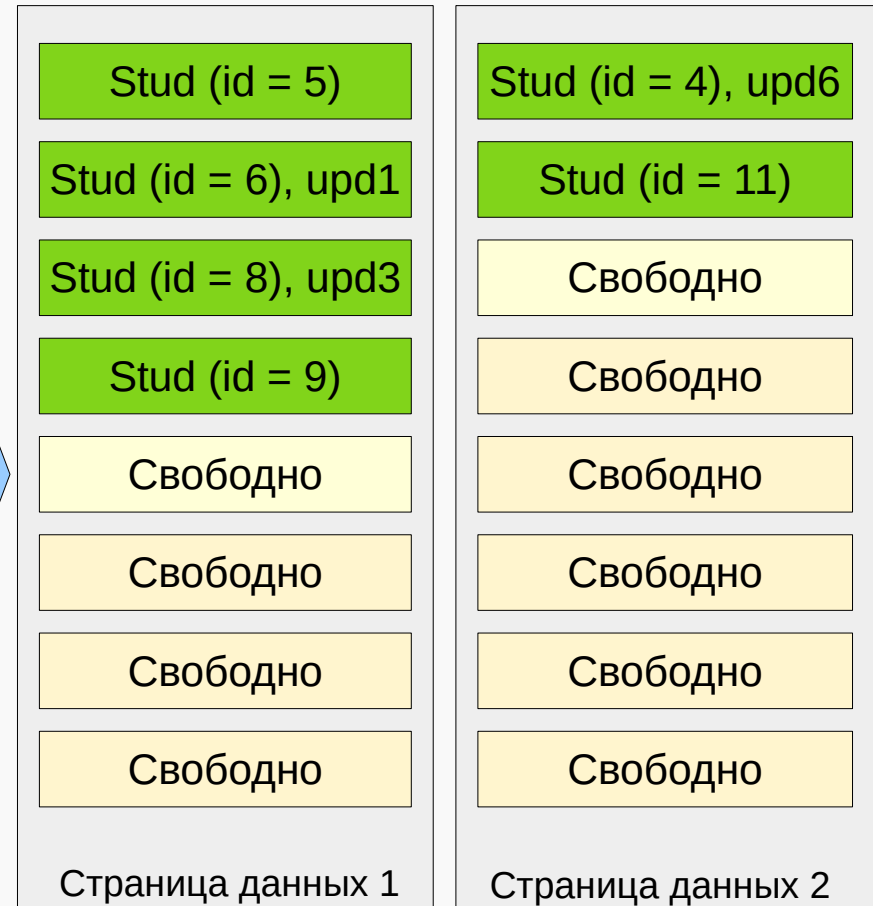
- Старые версии данных (мертвые записи) нужно удалять — они занимают место на диске.
- Для удаления таких записей используется VACUUM:
`VACUUM table[, table2, ...];`
- VACUUM:
 - По умолчанию;
 - `VACUUM FULL;`
 - `VACUUM FREEZE;`

VACUUM

До:

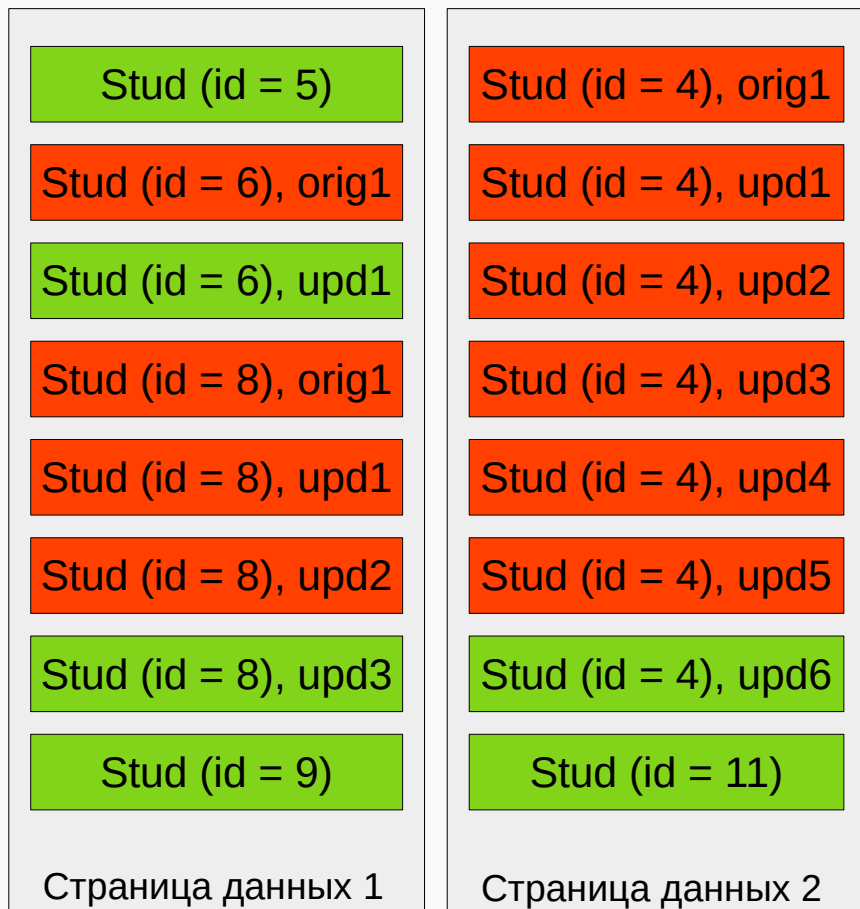


После VACUUM:

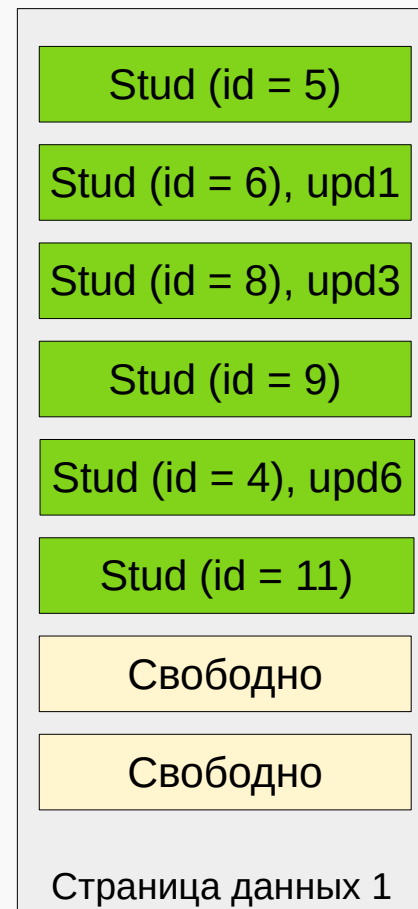
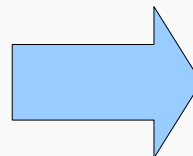


VACUUM FULL

До:



После VACUUM FULL:



autovacuum

- autovacuum = true (по умолчанию);
- меняется в postgresql.conf;
- autovacuum_vacuum_threshold
- autovacuum_vacuum_scale_factor
- autovacuum_cost_limit

Документация PostgreSQL:

<https://www.postgresql.org/docs/14/index.html>

Лицензия PostgreSQL:

PostgreSQL is released under the PostgreSQL License, a liberal Open Source license, similar to the BSD or MIT licenses.

PostgreSQL Database Management System
(formerly known as Postgres, then as Postgres95)

Portions Copyright © 1996-2022, The PostgreSQL Global Development Group

Portions Copyright © 1994, The Regents of the University of California

Permission to use, copy, modify, and distribute this software and its documentation for any purpose, without fee, and without a written agreement is hereby granted, provided that the above copyright notice and this paragraph and the following two paragraphs appear in all copies.

IN NO EVENT SHALL THE UNIVERSITY OF CALIFORNIA BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, INCLUDING LOST PROFITS, ARISING OUT OF THE USE OF THIS SOFTWARE AND ITS DOCUMENTATION, EVEN IF THE UNIVERSITY OF CALIFORNIA HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

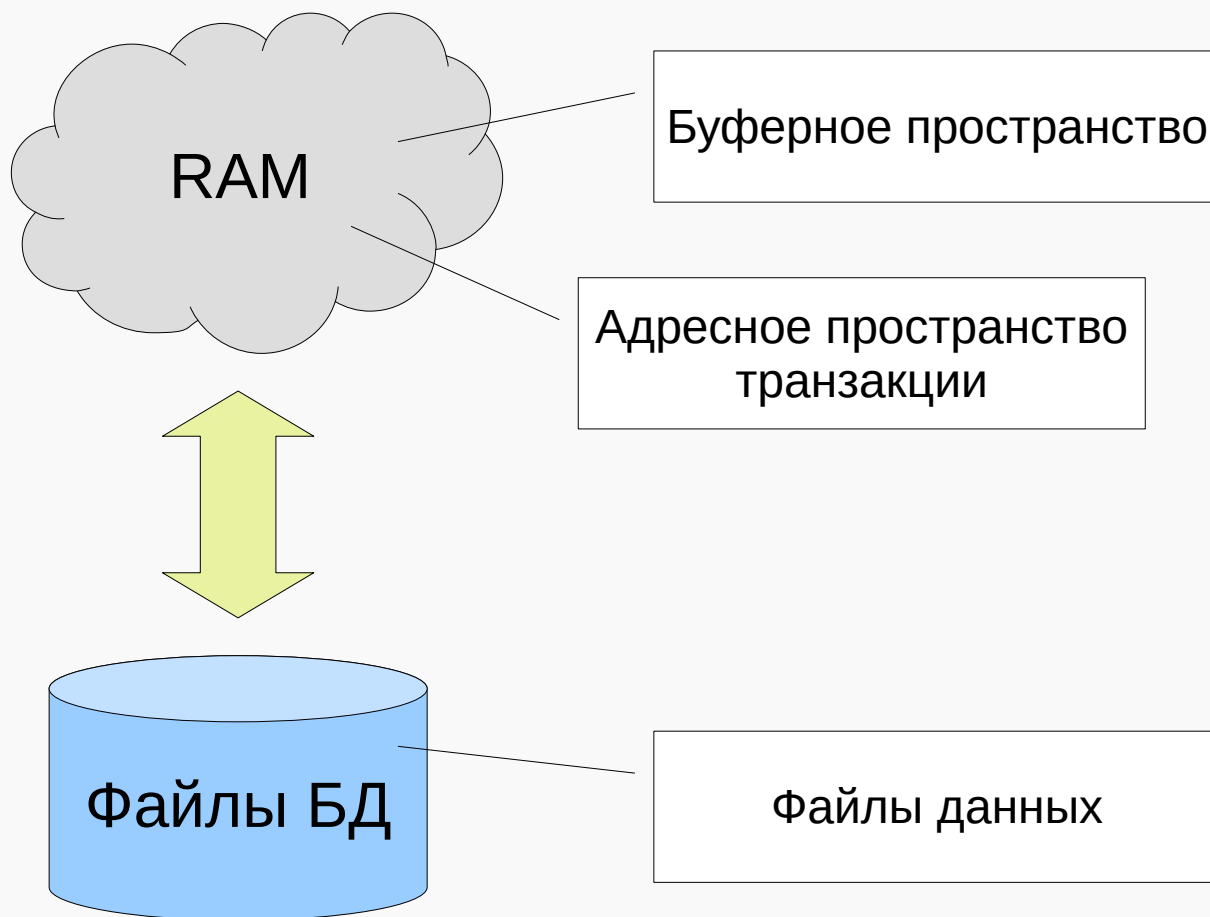
THE UNIVERSITY OF CALIFORNIA SPECIFICALLY DISCLAIMS ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE SOFTWARE PROVIDED HEREUNDER IS ON AN "AS IS" BASIS, AND THE UNIVERSITY OF CALIFORNIA HAS NO OBLIGATIONS TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

<https://www.interdb.jp/pg/index.html>

Лекция 5

1. Выполнение операций и восстановление данных

Память



Операции для синхронизации различных пространств памяти

- $M \leftarrow D(\text{Obj})$ — загрузить страницу, содержащую Obj , с диска в буферное пространство.
- $D \leftarrow M(\text{Obj})$ — сохранить страницу, содержащую Obj , на диск из буфера ОП.
- $T(v, T_x) \leftarrow M(\text{Obj})$ — копировать Obj в переменную v транзакции T_x :
 - если Obj не в M , то сначала $M \leftarrow D(\text{Obj})$.
- $M(\text{Obj}) \leftarrow T(v, T_x)$ — копировать значение v из транзакции T_x в Obj :
 - если Obj не в M , то сначала $M \leftarrow D(\text{Obj})$.
- **WriteLog** — сохранить лог на диск из соответствующего буфера.

```
SELECT * FROM ROOMS;
```

Room

Room_ID	Address	Room_Num	Size
1	Kronverksky, 49	374	20
2	Kronverksky, 49	375	25

Задача: переименовать аудитории

Room

Room_ID	Address	Room_Num	Size
1	Kronverksky, 49	374	20
2	Kronverksky, 49	375	25

BEGIN;

UPDATE ROOM SET Room_Num = 1331

WHERE Room_Num = 374;

UPDATE ROOM SET Room_Num = 1330

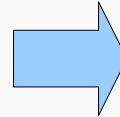
WHERE Room_Num = 375;

COMMIT;

Пример

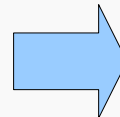
BEGIN;

```
UPDATE ROOM  
SET Room_Num = 1331  
WHERE Room_Num = 374;
```



$T(v_{\text{Room}}, Tx) \leftarrow M(\text{Room}_{374});$
 $V_{\text{Room}}(\text{Room_Num}) = 1331;$
 $M(\text{Room}_{374}) \leftarrow T(v_{\text{Room}}, Tx);$

```
UPDATE ROOM  
SET Room_Num = 1330  
WHERE Room_Num = 375;
```



$T(v_{\text{Room}}, Tx) \leftarrow M(\text{Room}_{375});$
 $V_{\text{Room}}(\text{Room_Num}) = 1330;$
 $M(\text{Room}_{375}) \leftarrow T(v_{\text{Room}}, Tx);$

COMMIT;

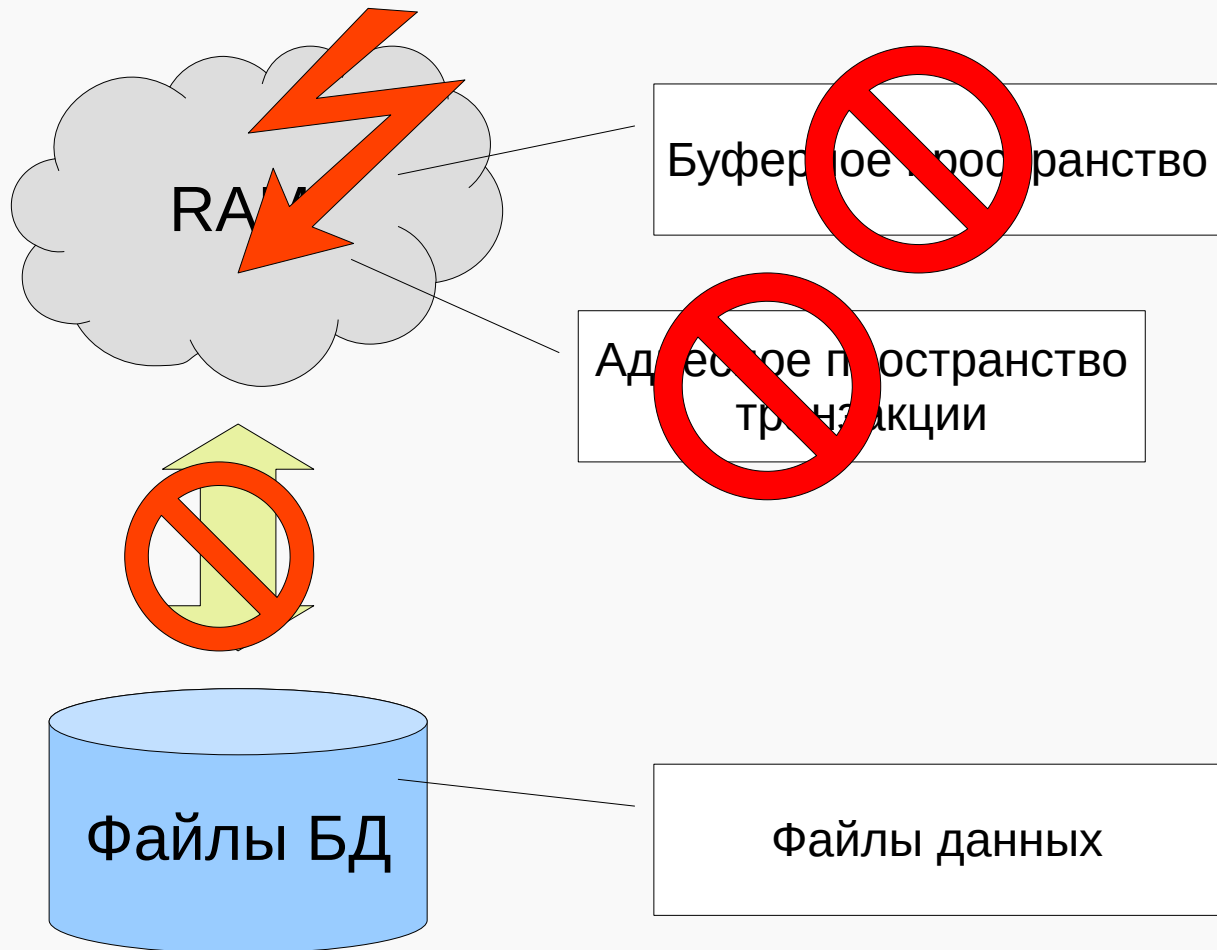
$D \leftarrow M(\text{Room}_{374}), M(\text{Room}_{375})$

*Выполнение операции
UPDATE сильно упрощено

Пример

Операция	Пам. тр.	Буферное пространство	Диск
$T(v_R, Tx) \leftarrow M(R_{374})$	374	$R_{374}=374, R_{375} = -$	$R_{374}=374, R_{375}=375$
$V_R(\text{Room_Num}) = 1331;$	1331	$R_{374}=374, R_{375} = -$	$R_{374}=374, R_{375}=375$
$M(R_{374}) \leftarrow T(v_R, Tx);$	1331	$R_{374}=1331, R_{375} = -$	$R_{374}=374, R_{375}=375$
$T(v_R, Tx) \leftarrow M(R_{375});$	375	$R_{374}=1331, R_{375}=375$	$R_{374}=374, R_{375}=375$
$V_R(\text{Room_Num}) = 1330;$	1330	$R_{374}=1331, R_{375}=375$	$R_{374}=374, R_{375}=375$
$M(R_{375}) \leftarrow T(v_R, Tx);$	1330	$R_{374}=1331, R_{375}=1330$	$R_{374}=374, R_{375}=375$
$D \leftarrow M(R_{374})$	1330	$R_{374}=1331, R_{375}=1330$	$R_{374}=1331, R_{375}=375$
$D \leftarrow M(R_{375})$	1330	$R_{374}=1331, R_{375}=1330$	$R_{374}=1331, R_{375}=1330$

«Незапланированный» отказ работы экземпляра



Пример

Операция	Пам. тр.	Буферное пространство	Диск
$T(v_R, Tx) \leftarrow M(R_{374})$	374	$R_{374}=374, R_{375} = -$	$R_{374}=374, R_{375}=375$
$V_R(\text{Room_Num}) = 1331;$	1331	$R_{374}=374, R_{375} = -$	$R_{374}=374, R_{375}=375$
$M(R_{374}) \leftarrow T(v_R, Tx);$	1331	$R_{374}=1331, R_{375} = -$	$R_{374}=374, R_{375}=375$
$T(v_R, Tx) \leftarrow M(R_{375});$	375	$R_{374}=1331, R_{375}=375$	$R_{374}=374, R_{375}=375$
$V_R(\text{Room_Num}) = 1330;$	1330	$R_{374}=1331, R_{375}=375$	$R_{374}=374, R_{375}=375$
$M(R_{375}) \leftarrow T(v_R, Tx);$	1330	$R_{374}=1331, R_{375}=1330$	$R_{374}=374, R_{375}=375$
$D \leftarrow M(R_{374})$	1330	$R_{374}=1331, R_{375}=1330$	$R_{374}=1331, R_{375}=375$
$D \leftarrow M(R_{375})$	1330	$R_{374}=1331, R_{375}=1330$	$R_{374}=1331, R_{375}=1330$

Если система «упадет» в данный момент — проблема!

2. UNDO-журнал

Логирование

- Для предотвращения ситуаций, связанных с появлением несогласованных данных можно использовать — **журнал** (лог).
- СУБД сохраняет информацию об изменениях и фиксациях данных в журнале:
 - первоначально запись сохраняется в соответствующем буфере (буфере журнала);
 - буфер журнала синхронизируется с файлами журнала на диске (WriteLog);

Журнал отмены

- Один из вариантов ведения журнала — журнал отмены, UNDO LOG

Возможные записи в журнале отмены:

- BEGIN Tx -- начало транзакции;
- Tx: R, old_val -- изменение: транзакция Tx
-- переписывает значение
-- old_val в R;
- COMMIT Tx -- успешное завершение транзакции;
- ABORT Tx -- преждевременное завершение транз-ии;
- START CHKn: $T_n, \dots, T_m$ -- начало создания контр. точки
- END CHKn -- завершение создания контр. точки

Журнал отмены

Правила записи в UNDO LOG:

- Запись изменения данных (Tx: R, old_val) записывается в журнал (на диск) ДО сохранения обновленного значения на диске.
- COMMIT Tx должен быть добавлен в журнал (на диск) **ПОСЛЕ** обновления всех файлов данных, связанных с изменениями данной транзакции (после синхронизации буферного пространства данных с файлами данных).

Пример

Операция	Диск	Журнал
		BEGIN T1;
$T(v_R, Tx) \leftarrow M(R_{374})$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1331;$	$R_{374}=374, R_{375}=375$	
$M(R_{374}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: $R_{374}, 374$
$T(v_R, Tx) \leftarrow M(R_{375});$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1330;$	$R_{374}=374, R_{375}=375$	
$M(R_{375}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: $R_{375}, 375$
WriteLog		-- записи на диске
$D \leftarrow M(R_{374})$	$R_{374}=1331, R_{375}=375$	
$D \leftarrow M(R_{375})$	$R_{374}=1331, R_{375}=1330$	
		COMMIT T1
WriteLog		-- COMMIT на диске

Использование журнала отмены для восстановления данных (1)

Действия при восстановлении данных (1):

- СУБД сканирует журнал от новых записей до старых. Отдельно фиксируются транзакции:
 - Группа 1: для которых есть запись COMMIT Tx;
 - Группа 2: транзакции для которых есть запись ABORT Tx, незавершенные транзакции.
- Когда в журнале встречается запись изменения (Tx: R, old_val), анализируется Tx. Если Tx:
 - из группы 1 действия не предпринимаются.
 - из группы 2: для R восстанавливается old_val (происходит отмена изменений).

Использование журнала отмены для восстановления данных (2)

Действия при восстановлении данных (2):

- СУБД записывает ABORT Tx для каждой незавершенной транзакции из группы 2.
- Запускается операция WriteLog, чтобы записи вида ABORT Tx оказались в журнале.

Пример (UNDO LOG)

Операция	Диск	Журнал
		BEGIN T1;
$T(v_R, Tx) \leftarrow M(R_{374})$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1331;$	$R_{374}=374, R_{375}=375$	
$M(R_{374}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: $R_{374}, 374$
$T(v_R, Tx) \leftarrow M(R_{375});$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1330;$	$R_{374}=374, R_{375}=375$	
$M(R_{375}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: $R_{375}, 375$
WriteLog		-- записи на диске
$D \leftarrow M(R_{374})$	$R_{374}=1331, R_{375}=375$	

Произошел сбой!

Пример (UNDO LOG)

Операция	Диск	Журнал
		BEGIN T1;
$T(v_R, Tx) \leftarrow M(R_{374})$	$R_{374}=374, R_{375}=375$	Возвращаем первоначальное значение
$V_R(\text{Room_Num}) = 1331;$	$R_{374}=374, R_{375}=375$	
$M(R_{374}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: $R_{374}, 374$
$T(v_R, Tx) \leftarrow M(R_{375});$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1330;$	$R_{374}=374, R_{375}=375$	
$M(R_{375}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: $R_{375}, 375$
WriteLog		-- записи на диске
$D \leftarrow M(R_{374})$	$R_{374}=1331, R_{375}=375$	

Возвращаем первоначальное значение

Контрольные точки

- Для восстановления данных нужна какая-то стартовая точка, чтобы не анализировать транзакции с самого начала работы с БД.
- Контрольная точка (checkpoint) — точка синхронизации, во время которой происходит синхронизация данных из буферов с соответствующими файлами на диске.

Контрольные точки

Создание контрольной точки не одномоментно:

- 1) В журнал добавляется START CHKn: $T_n, \dots, T_m$, фиксируется на диске через WriteLog.
- 2) Ожидание завершения транзакций, которые работали в момент создания контрольной точки ($T_n, \dots, T_m$). В это время могут начинаться другие транзакции.
- 3) В журнал добавляется END CHKn, фиксируется на диске через WriteLog.
- 4) Записи перед START CHKn, у которой есть END CHKn, могут быть удалены.

UNDO LOG и контрольные точки

Действия при восстановлении данных (3):

- СУБД сканирует журнал и встречает запись END CHKn: $T_n, \dots, T_m$ (а не START CHKn):
 - Значит установка контрольной точки n завершена;
 - Сканирование журнала до записи START CHKn, так как незавершенные транзакции могут быть только после записи START CHKn:
 - × среди транзакций, которые начали работу после начала установки контрольной точки.
 - Остальные действия как раньше.

UNDO LOG и контрольные точки

Действия при восстановлении данных (4):

- СУБД сканирует журнал и встречает запись START CHKn: $T_n, \dots, T_m$ (а не END CHKn):
 - Значит установка контрольной точки n не завершена (проблема во время установки);
 - Сканирование журнала до первой записи из самой ранней транзакции среди $T_n, \dots, T_m$:
 - × такая запись будет до START CHKn;
 - Остальные действия как раньше.

Недостатки UNDO LOG

- Нельзя завершить транзакцию (зафиксировать COMMIT в журнале) до записи изменений данных в файлы данных.
- Много операций ввода-вывода с диском, так как надо синхронизировать данные для каждой транзакции.

Лекция 6. Часть II

1. REDO-журнал

Журнал повторений

- Другой способ ведения журнала — журнал повторений, REDO LOG (повторяются завершённые транзакции).

Возможные записи в журнале повторений:

- BEGIN Tx -- начало транзакции;
- Tx: R, new_val -- изменение: транзакция Tx
 -- изменяет значение
 -- на new_val в R;
- COMMIT Tx -- успешное завершение транзакции;
- ABORT Tx -- преждевременное завершение транз-ии;
- START CHKn: $T_n, \dots, T_m$ -- начало создания контр. точки
- END CHKn -- завершение создания контр. точки

Журнал повторений

Правило записи в REDO LOG (правило Write Ahead Logging):

- Запись изменения данных (Tx: R, new_val) записывается в журнал (на диск) **ДО** сохранения обновленного значения на диске.
- COMMIT Tx должен быть добавлен в журнал (на диск) **ДО** обновления любого из файлов данных, связанных с изменениями данной транзакции (до синхронизации буферного пространства данных с файлами данных).
- То есть в WAL (на диске) вносятся записи изменения, которые произошли в Tx, и запись COMMIT Tx, и только после этого — идет синхронизация буферов с диском.

Есть ли на слайде ошибка?

Операция	Данные на диске	Журнал (WAL)
		BEGIN T1;
$T(v_R, Tx) \leftarrow M(R_{374})$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1331;$	$R_{374}=374, R_{375}=375$	
$M(R_{374}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: $R_{374}, 374$
$T(v_R, Tx) \leftarrow M(R_{375});$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1330;$	$R_{374}=374, R_{375}=375$	
$M(R_{375}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: $R_{375}, 375$
		COMMIT T1
WriteLog		
$D \leftarrow M(R_{374}), D \leftarrow M(R_{375})$	$R_{374}=1331, R_{375}=1330$	

Пример

Операция	Данные на диске	Журнал (WAL)
		BEGIN T1;
$T(v_R, Tx) \leftarrow M(R_{374})$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1331;$	$R_{374}=374, R_{375}=375$	
$M(R_{374}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{374} , 1331
$T(v_R, Tx) \leftarrow M(R_{375});$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1330;$	$R_{374}=374, R_{375}=375$	
$M(R_{375}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{375} , 1330
		COMMIT T1
WriteLog		
$D \leftarrow M(R_{374}), D \leftarrow M(R_{375})$	$R_{374}=1331, R_{375}=1330$	

Лог уже на диске, а буферы не синхронизированы с файлами данных

Использование журнала повторений для восстановления данных (1)

Действия при восстановлении данных (1):

- СУБД сканирует журнал от **старых записей до новых**. Отдельно фиксируются транзакции:
 - Группа 1: для которых есть запись COMMIT Tx;
 - Группа 2: транзакции для которых есть запись ABORT Tx, незавершенные транзакции.

Использование журнала повторений для восстановления данных (2)

Действия при восстановлении данных (2):

- Когда в журнале встречается запись изменения (Tx: R, old_val), анализируется Tx. Если Tx:
 - из группы 1: для R перезаписывается new_val (происходит «повтор», перезапись изменений).
 - из группы 2:
 - СУБД записывает ABORT Tx для каждой незавершенной транзакции из группы 2 (у которой не было ABORT).
 - Запускается операция WriteLog, чтобы записи вида ABORT Tx оказались в журнале.

Пример 1 (REDO LOG)

Операция	Данные на диске	Журнал (WAL)
		BEGIN T1;
$T(v_R, Tx) \leftarrow M(R_{374})$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1331;$	$R_{374}=374, R_{375}=375$	
$M(R_{374}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{374} , 1331
$T(v_R, Tx) \leftarrow M(R_{375});$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1330;$	$R_{374}=374, R_{375}=375$	
$M(R_{375}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{375} , 1330



Произошел сбой!

Действия для восстановления?

Операция	Данные на диске	Журнал (WAL)
		BEGIN T1;
$T(v_R, Tx) \leftarrow M(R_{374})$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1331;$	$R_{374}=374, R_{375}=375$	
$M(R_{374}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{374} , 1331
$T(v_R, Tx) \leftarrow M(R_{375});$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1330;$	$R_{374}=374, R_{375}=375$	
$M(R_{375}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{375} , 1330



Произошел сбой!

Восстановление

Операция	Данные на диске	Журнал (WAL)
		BEGIN T1;
$T(v_R, Tx) \leftarrow M(R_{374})$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1331;$	$R_{374}=374, R_{375}=375$	
$M(R_{374}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{374} , 1331
$T(v_R, Tx) \leftarrow M(R_{375});$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1330;$	$R_{374}=374, R_{375}=375$	
$M(R_{375}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{375} , 1330

Нет COMMIT: транзакция
не завершена. ABORT.

Пример 2 (REDO LOG)

Операция	Данные на диске	Журнал (WAL)
		BEGIN T1;
$T(v_R, Tx) \leftarrow M(R_{374})$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1331;$	$R_{374}=374, R_{375}=375$	
$M(R_{374}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{374} , 1331
$T(v_R, Tx) \leftarrow M(R_{375});$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1330;$	$R_{374}=374, R_{375}=375$	
$M(R_{375}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{375} , 1330
		COMMIT T1
WriteLog		

Произошел сбой!

Действия для восстановления?

Операция	Данные на диске	Журнал (WAL)
		BEGIN T1;
$T(v_R, Tx) \leftarrow M(R_{374})$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1331;$	$R_{374}=374, R_{375}=375$	
$M(R_{374}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{374} , 1331
$T(v_R, Tx) \leftarrow M(R_{375});$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1330;$	$R_{374}=374, R_{375}=375$	
$M(R_{375}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{375} , 1330
		COMMIT T1
WriteLog		

Произошел сбой!

Восстановление

Операция	Данные на диске	Журнал (WAL)
		BEGIN T1;
$T(v_R, Tx) \leftarrow M(R_{374})$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1331;$	$R_{374}=374, R_{375}=375$	
$M(R_{374}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{374} , 1331
$T(v_R, Tx) \leftarrow M(R_{375});$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1330;$	$R_{374}=374, R_{375}=375$	
$M(R_{375}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{375} , 1330
		COMMIT T1
WriteLog		



Транзакция COMMIT и
зафиксирована → повторяем сначала

Пример 1 (REDO LOG)

Операция	Данные на диске	Журнал (WAL)
		BEGIN T1;
$T(v_R, Tx) \leftarrow M(R_{374})$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1331;$	$R_{374}=374, R_{375}=375$	
$M(R_{374}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{374} , 1331
$T(v_R, Tx) \leftarrow M(R_{375});$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1330;$	$R_{374}=374, R_{375}=375$	
$M(R_{375}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{375} , 1330
		COMMIT T1



Произошел сбой!

Действия для восстановления?

Операция	Данные на диске	Журнал (WAL)
		BEGIN T1;
$T(v_R, Tx) \leftarrow M(R_{374})$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1331;$	$R_{374}=374, R_{375}=375$	
$M(R_{374}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{374} , 1331
$T(v_R, Tx) \leftarrow M(R_{375});$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1330;$	$R_{374}=374, R_{375}=375$	
$M(R_{375}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{375} , 1330
		COMMIT T1

Произошел сбой!

Восстановление

Операция	Данные на диске	Журнал (WAL)
		BEGIN T1;
$T(v_R, Tx) \leftarrow M(R_{374})$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1331;$	$R_{374}=374, R_{375}=375$	
$M(R_{374}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{374} , 1331
$T(v_R, Tx) \leftarrow M(R_{375});$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1330;$	$R_{374}=374, R_{375}=375$	
$M(R_{375}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{375} , 1330
		COMMIT T1

Зависит от того попал ли в WAL на диске COMMIT. WriteLog мог выполнить другой процесс

Контрольные точки (REDO LOG)

Создание контрольной точки не одномоментно:

- 1) В журнал добавляется START CHKn: $T_n, \dots, T_m$, фиксируется на диске через WriteLog.
- 2) Сохранение всех изменений («грязных страниц») на диск, измененных завершенными транзакциями на момент START CHKn.
- 3) В журнал добавляется END CHKn, фиксируется на диске через WriteLog.

REDO LOG и контрольные точки

Действия при восстановлении данных (3):

- СУБД сканирует журнал в поисках данных о последней контрольной точке и встречается запись END CHKn (а не START CHKn: $T_n, \dots, T_m$):
 - Остальные действия как раньше, но только для транзакций ($T_n, \dots, T_m$) или тех, которые были начаты после записи START CHKn: $T_n, \dots, T_m$.

REDO LOG и контрольные точки

Действия при восстановлении данных (4):

- СУБД сканирует журнал в поисках данных о последней контрольной точке и встречает запись START CHKn: T_n , ..., T_m (а не END CHKn):
 - Значит установка контрольной точки n не завершена (проблема во время установки);
 - Ищем END CHKn-1, дальнейшие действия, как на пред. слайде, только для CHKn-1.

Недостатки REDO LOG

- Нельзя «сбросить» данные из буферов на диск до завершения транзакции → требуется много памяти под буферы.

2. UNDO/REDO-журнал

Журнал отмены/повторений

- Еще один способ — журнал отмены/повторений, UNDO/REDO LOG.

Возможные записи в журнале повторений:

- BEGIN Tx -- начало транзакции;
- Tx: R, **old_val**, **new_val** -- изменение: транзакция Tx
-- изменяет значение
-- на new_val в R;
- COMMIT Tx -- успешное завершение транзакции;
- ABORT Tx -- преждевременное завершение транз-ии;
- START CHKn: $T_n, \dots, T_m$ -- начало создания контр. точки
- END CHKn -- завершение создания контр. точки

Журнал отмены/повторений

Правило записи в UNDO/REDO LOG:

- Запись изменения данных (Tx: R, old_val, new_val) записывается в журнал (на диск) **ДО** сохранения обновленного значения на диске.
- COMMIT Tx может быть добавлен в журнал (на диск) как до, так и после обновления любого из файлов данных, связанных с изменениями данной транзакции.

Пример

Операция	Данные на диске	Журнал (WAL)
		BEGIN T1;
$T(v_R, Tx) \leftarrow M(R_{374})$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1331;$	$R_{374}=374, R_{375}=375$	
$M(R_{374}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{374} , 1331
$T(v_R, Tx) \leftarrow M(R_{375});$	$R_{374}=374, R_{375}=375$	
$V_R(\text{Room_Num}) = 1330;$	$R_{374}=374, R_{375}=375$	
$M(R_{375}) \leftarrow T(v_R, Tx);$	$R_{374}=374, R_{375}=375$	T1: R_{375} , 1330
WriteLog		
$D \leftarrow M(R_{374}), D \leftarrow M(R_{375})$	$R_{374}=1331, R_{375}=1330$	
		COMMIT T1
WriteLog		

Использование UNDO/REDO для восстановления данных

Действия при восстановлении данных:

- повторить операции для завершенных транзакций (от ранних к поздним);
- отменить операции для незавершенных транзакций (от поздних к ранним);
- запись COMMIT должна быть сразу синхронизирована с журналом;

Контрольные точки (UNDO/REDO LOG)

Создание контрольной точки не одномоментно:

- 1) В журнал добавляется START CHKn: $T_n, \dots, T_m$, фиксируется на диске через WriteLog.
- 2) Сохранение всех изменений («грязных страниц») на диск.
 - Если транзакция может прервать работу (ROLLBACK), ее изменения не должны фиксироваться в буферах данных.
- 3) В журнал добавляется END CHKn, фиксируется на диске через WriteLog.

UNDO/REDO LOG и контрольные точки

Действия при восстановлении данных (2):

- Состоит из шагов для UNDO и REDO.
- Благодаря тому, что при установке контрольной точки происходит сохранение всех изменений («грязных страниц») на диск, REDO нужно возвращаться только к последней START CHKn, для которой есть END CHKn:
 - если на момент START CHKn были незавершенные транзакции, их изменения зафиксированы при создании контрольной точки:
 - × Если транзакция в итоге завершиться успешно — REDO (с момента начала этой контр. точки);
 - × Если неуспешно — UNDO — как обычно.

steal/no-steal, force/no-force

- **no-steal** — подход, при котором страница из буфера данных не может быть записана на диск до завершения (COMMIT) транзакции, в ходе которой она была изменена.
- **steal** — если позволено записывать в файлы данных данные еще незавершенных транзакций (для оптимизации запросов к диску или если буферная память заканчивается).
- **force** — если требуется, чтобы все страницы, измененные транзакцией, записывались сразу перед COMMIT.
- **no-force** — изменения, осуществленные завершенными транзакциями могут быть не зафиксированы на диске.

ARIES (Algorithms for Recovery and Isolation Exploiting Semantics) — алгоритм для восстановления данных.

Базируется на:

- WAL;
- steal;
- no-force;

ARIES происходит в 3 этапа (3 фазы):

- **Анализ** — поиск «грязных» страниц и незавершенных транзакций;
- **REDO** — повторение всех действий;
- **UNDO** — откат действий незавершенных транзакций;

Для работы требуется LSN, таблица транзакций, таблица «грязных страниц».

Документация PostgreSQL:

<https://www.postgresql.org/docs/14/index.html>

Лицензия PostgreSQL:

PostgreSQL is released under the PostgreSQL License, a liberal Open Source license, similar to the BSD or MIT licenses.

PostgreSQL Database Management System
(formerly known as Postgres, then as Postgres95)

Portions Copyright © 1996-2022, The PostgreSQL Global Development Group

Portions Copyright © 1994, The Regents of the University of California

Permission to use, copy, modify, and distribute this software and its documentation for any purpose, without fee, and without a written agreement is hereby granted, provided that the above copyright notice and this paragraph and the following two paragraphs appear in all copies.

IN NO EVENT SHALL THE UNIVERSITY OF CALIFORNIA BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, INCLUDING LOST PROFITS, ARISING OUT OF THE USE OF THIS SOFTWARE AND ITS DOCUMENTATION, EVEN IF THE UNIVERSITY OF CALIFORNIA HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE UNIVERSITY OF CALIFORNIA SPECIFICALLY DISCLAIMS ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE SOFTWARE PROVIDED HEREUNDER IS ON AN "AS IS" BASIS, AND THE UNIVERSITY OF CALIFORNIA HAS NO OBLIGATIONS TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

<https://www.interdb.jp/pg/index.html>

Лекция 7

Лектор:
Николаев В.В.

1. Резервное копирование

Терминология

- Стратегия резервного копирования бывает:
 - полная — охватывает всю базу данных;
 - частичная — охватывает только часть базы данных.
- Резервная копия может содержать:
 - все страницы в пределах выбранных файлов;
 - ту информацию, которая изменилась с момента последнего резервного копирования (инкрементная):
 - кумулятивная (изменения — до последнего уровня 0);
 - дифференциальная (изменения — до последнего инкрементного резервного копирования).

Виды резервного копирования (1)

Виды резервного копирования:

- «холодное» — создание резервной копии, когда экземпляр сервера БД остановлен;
- «горячее» — создание резервной копии во время работы экземпляра сервера БД;

Виды резервного копирования (2)

Виды резервного копирования:

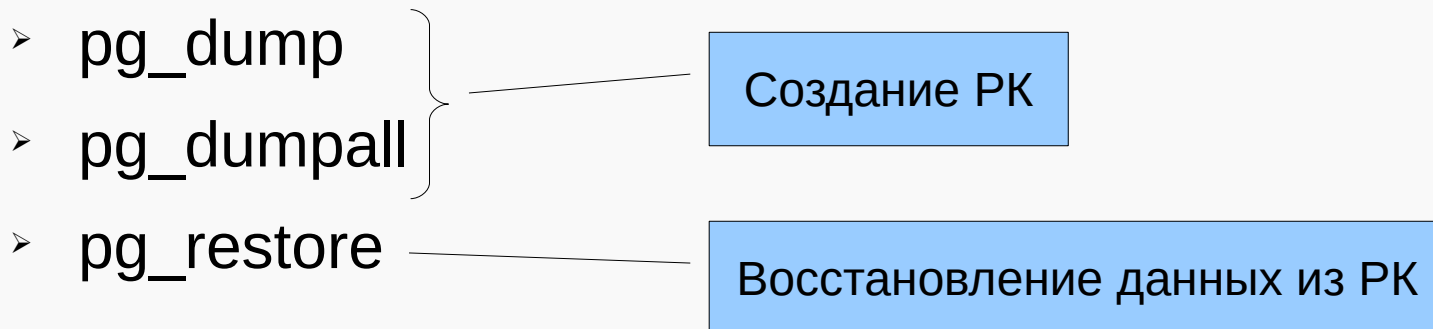
- логическое;
- физическое;

2. Логическое РК

- Копия объектов БД и структур данных создается в виде команд/предложений языка SQL, сами файлы не копируются.
- Утилиты в PostgreSQL для осуществления логического резервного копирования/восстановления:
 - pg_dump
 - pg_dumpall
 - pg_restore

Логическое РК

- Копия объектов БД и структур данных создается в виде команд/предложений языка SQL, сами файлы не копируются.
- Утилиты в PostgreSQL для осуществления логического резервного копирования/восстановления:



pg_dump

- Служит для создания резервной копии БД или ее части.
- В общем случае создается SQL-файл с командами, которые позволяют воссоздать логическое состояние БД на момент создания бэкапа.
- Использование:

`pg_dump mydb > mydbdump`

Имя БД

Файл с копией

`pg_dump -U postgres1 -f mydbdump mydb`

pg_dump: форматы РК

- **plain text**, текстовый формат — в файле ~SQL-предложения:
 - можно восстановить, выполнив в psql;
- **custom** формат:
 - возможно сжатие (если есть zlib);
 - бинарный файл;
- **в виде директории** — один файл для каждой ~таблицы;
- **tar** — архив РК, в архиве содержимое, как для формата в виде директории;

pg_dump: форматы РК

- **plain text:**

`pg_dump -f mydbdump mydb`

- **custom** формат:

`pg_dump -Fc -f mydbdump mydb`

- **в виде директории:**

`pg_dump -Fd Student -f mydbdump mydb`

- **tar:**

`pg_dump -Ft Student -f mydbdump mydb`

`pg_restore`

Формат РК в виде директории

`pg_dump -Fd Student -f mydbdump mydb`

- Для отдельных объектов БД создаются отдельные файлы:
 - возможно сжатие файлов в директории;
- В директории есть файл `toc.dat`:
 - индекс для `pg_restore`;
 - для поиска файлов внутри директории;

Текстовый формат

- Используется по умолчанию.
- Содержимое — предложения для восстановления логического состояния БД.
- По умолчанию для **добавления данных** используется PostgreSQL команда **COPY**.
- Для генерации INSERT:

```
pg_dump -U postgres1 --inserts -f mydbdump mydb
```

```
pg_dump -U postgres1 --column-inserts -f mydbdump  
mydb
```

Особенности использования pg_dump

- По умолчанию не создаются предложения для создания БД.
- Чтобы восстановить данные нужно подключиться к существующей БД.
- Для добавления создания БД (CREATE DATABASE) можно использовать опцию `—create (-C)`:
 - при использовании опции в бэкап добавляются предложения для подключения к созданной БД:
`pg_dump -U postgres1 --create -f mydbdump mydb`

Получение скрипта для восстановления части БД

Можно задать создание скрипта для восстановления только определенных таблиц, структуры БД, содержимого БД:

- `pg_dump -t Student -f mydbdump mydb -- only Student`
- `pg_dump -T Student -f mydbdump mydb -- all except
-- Student`
- `pg_dump -a -f mydbdump mydb -- only data`
- `pg_dump -s -f mydbdump mydb -- only schema (DDL)`

Восстановление БД

- Использование:

```
psql mydb < mydbdump
```

- Используемые в бэкапе пользователи, роли должны существовать/быть воссозданы до запуска скрипта.

Ошибки при восстановлении БД

- Чтобы остановить скрипт после возникновения первой ошибки (невыполненного предложения):

```
psql --set ON_ERROR_STOP=on mydb < mydbdump
```

Результаты прошедших команд останутся в случае возникновения ошибки

- Можно указать, чтобы восстановление выполнялось в рамках **одной транзакции**:

```
psql --single-transaction mydb < mydbdump
```

pg_restore

- Утилита для восстановления данных на основе скриптов, созданных pg_dump:

```
pg_restore -d mydb mydbdump
```

- Можно посмотреть список команд, которые будут исполнены при восстановлении БД:

```
pg_restore mydbdump -f test.sql
```

- Если используется директория — можно включить свой toc-файл и сделать частичное восстановление.

pg_dumpall

- Утилита для создания скриптов восстановления всех БД в кластере:

```
pg_dumpall -U postgres1 -f mydbdump
```

- Создает SQL-скрипт.
- Кроме БД сохраняются глобальные объекты (пользователи, роли, табличные пространства).
- Восстановление БД:

```
psql -f dumpfile postgres
```

Осуществление резервного копирования

- Возможен параллельный режим:
pg_dump, опция -j:
 - можно указать число параллельных процессов.
- Есть готовые скрипты-шаблоны для организации резервного копирования:

https://wiki.postgresql.org/wiki/Automated_Backup_on_Linux

3. Физическое РК

- При создания резервной копии происходит копирование файлов исходного кластера.
- Утилиты в PostgreSQL для осуществления физического резервного копирования/восстановления:
 - pg_basebackup
 - функции pg_start_backup, pg_stop_backup

Резервное копирование на уровне файловой системы

- Заключается в полном копировании директории кластера БД средствами файловой системы.
- Обычно производится в «холодном режиме» - сервер БД должен быть остановлен:
 - «горячий режим» возможен если файловая система поддерживает возможность получения единовременного «снимка» данных.
- Может быть произведен только для всех объектов БД.

Непрерывное архивирование

- Базируется на существовании WAL (PGDATA/pg_wal):
 - состояние БД может быть восстановлено путем повтора зафиксированных в логе операций;
 - конфигурационные файлы не восстанавливаются через WAL (postgresql.conf, ...);

Резервная копия = полная копия + WAL-файлы

Физическое РК + WAL

- БД не обязательно должна быть в полностью согласованном состоянии:
 - для приведения состояния к согласованному может использоваться WAL;
 - С точки зрения работы WAL — механизмы аналогичны тем, которые используются при восстановлении после сбоев.
- Можно восстановить данные в одно из временных состояний между снятием бэкапа и последним WAL.
- Можно восстановить базу данных целиком (а не отдельную часть).

Этапы для реализации непрерывного архивирования

- 1) Организация архивирования WAL.
- 2) Создание базовой резервной копии.

Этап 1: архивирование WAL

- На диске WAL разделен на **сегменты** по 16 Мб (по умолчанию).
- Необходимо осуществить копирование добавляемых сегментов на внешнее хранилище.
- Для включения возможности резервного копирования:
 - `archive_mode = on`
 - `wal_level = replica` # минимум
- PostgreSQL позволяет задать команду для создания копий WAL по мере их создания (`postgresql.conf`):
 - `archive_command = ' some command '`

Команда для архивирования

Задается в postgresql.conf:

archive_command = 'cp %p /path2/%f' # Unix

- %p — заменяется на полное имя файла для архив-я.
- %f — заменяется на имя файла.

Назначение:

- копирование сегментов WAL в path2.
- команда должна возвращать 0 только в случае успешного завершения.

archive_command

- Значение можно изменить, перегрузив значения конфигурационного файла.
- Частоту создания новых сегментов можно регулировать через `archive_timeout`:
 - если он небольшой будет создаваться много незаполненных сегментов и тратиться лишнее место.

Этап 2: создание базовой резервной копии - pg_basebackup

- Можно осуществить через pg_basebackup:

pg_basebackup -D /path/backup -T /old/ts=\$(pwd)/new/ts

- backup_history-файл в pg_wal (***.backup) — имя содержит название первого WAL-сегмента для осуществления восстановления.
- С помощью pg_basebackup можно осуществить создание обособленной копии (без осуществления непрерывного копирования).

Этап 2: создание базовой резервной копии — через pg*_backup

функции pg_start_backup, pg_stop_backup:

- SELECT pg_start_backup('my_label');
 - создается **backup_my_label** файл — содержит различную информацию о резервной копии (время начала, метку, ...);
 - создается tablespace_map;
 - создается контрольная точка;
- пользователем производится копирование файлов кластера БД;
- SELECT pg_stop_backup();

Восстановление данных на основе бэкапа с непрерывным архивированием (1)

- 1) Нужно остановить сервер БД.
- 2) Если возможно, сделать копию — хотя бы `pg_wal` (на случай, если есть незаархивированные WAL).
- 3) Перед шагом: **проверить наличие бэкапа**. Если бэкап есть — удалить содержимое PGDATA и, если есть, внешних директорий (с табличными пространствами и т. д.).
- 4) Скопировать в PGDATA и внешние директории данные ранее полученного бэкапа.
- 5) Очистить директорию PGDATA/`pg_wal`.
- 6) Если есть незаархивированные WAL (с шага 2) — поместить их в PGDATA/`pg_wal`.

Восстановление данных на основе бэкапа с непрерывным архивированием (2)

7) Установить настройки для восстановления в `postgresql.conf` (**`restore_command`**, должна возвращать ненулевой код в случае неудачи) + запретить внешние подключения (для пользователей) в `pg_hba.conf`.

8) Создать файл **`recovery.signal`** — говорит серверу, что надо стартовать в режиме восстановления (`PGDATA/data`).

9) Запустить сервер → он перейдет в режим восстановления. По завершении `recovery.signal` будет удален.

10) Разрешить подключения для пользователей в `pg_hba.conf`.

restore_command

- Задает логику получения архивированных WAL-сегментов:

```
restore_command = 'cp /path/%f %p'
```

- %f - имя файла (WAL-сегмента);
- %p - путь, куда копировать файлы (WAL-сегменты).
- Сегменты, которые не были найдены в архиве, будут искаться в pg_wal:
 - приоритет тем сегментам, которые в архиве.

Документация PostgreSQL:

<https://www.postgresql.org/docs/14/index.html>

Лицензия PostgreSQL:

PostgreSQL is released under the PostgreSQL License, a liberal Open Source license, similar to the BSD or MIT licenses.

PostgreSQL Database Management System
(formerly known as Postgres, then as Postgres95)

Portions Copyright © 1996-2022, The PostgreSQL Global Development Group

Portions Copyright © 1994, The Regents of the University of California

Permission to use, copy, modify, and distribute this software and its documentation for any purpose, without fee, and without a written agreement is hereby granted, provided that the above copyright notice and this paragraph and the following two paragraphs appear in all copies.

IN NO EVENT SHALL THE UNIVERSITY OF CALIFORNIA BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, INCLUDING LOST PROFITS, ARISING OUT OF THE USE OF THIS SOFTWARE AND ITS DOCUMENTATION, EVEN IF THE UNIVERSITY OF CALIFORNIA HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE UNIVERSITY OF CALIFORNIA SPECIFICALLY DISCLAIMS ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE SOFTWARE PROVIDED HEREUNDER IS ON AN "AS IS" BASIS, AND THE UNIVERSITY OF CALIFORNIA HAS NO OBLIGATIONS TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

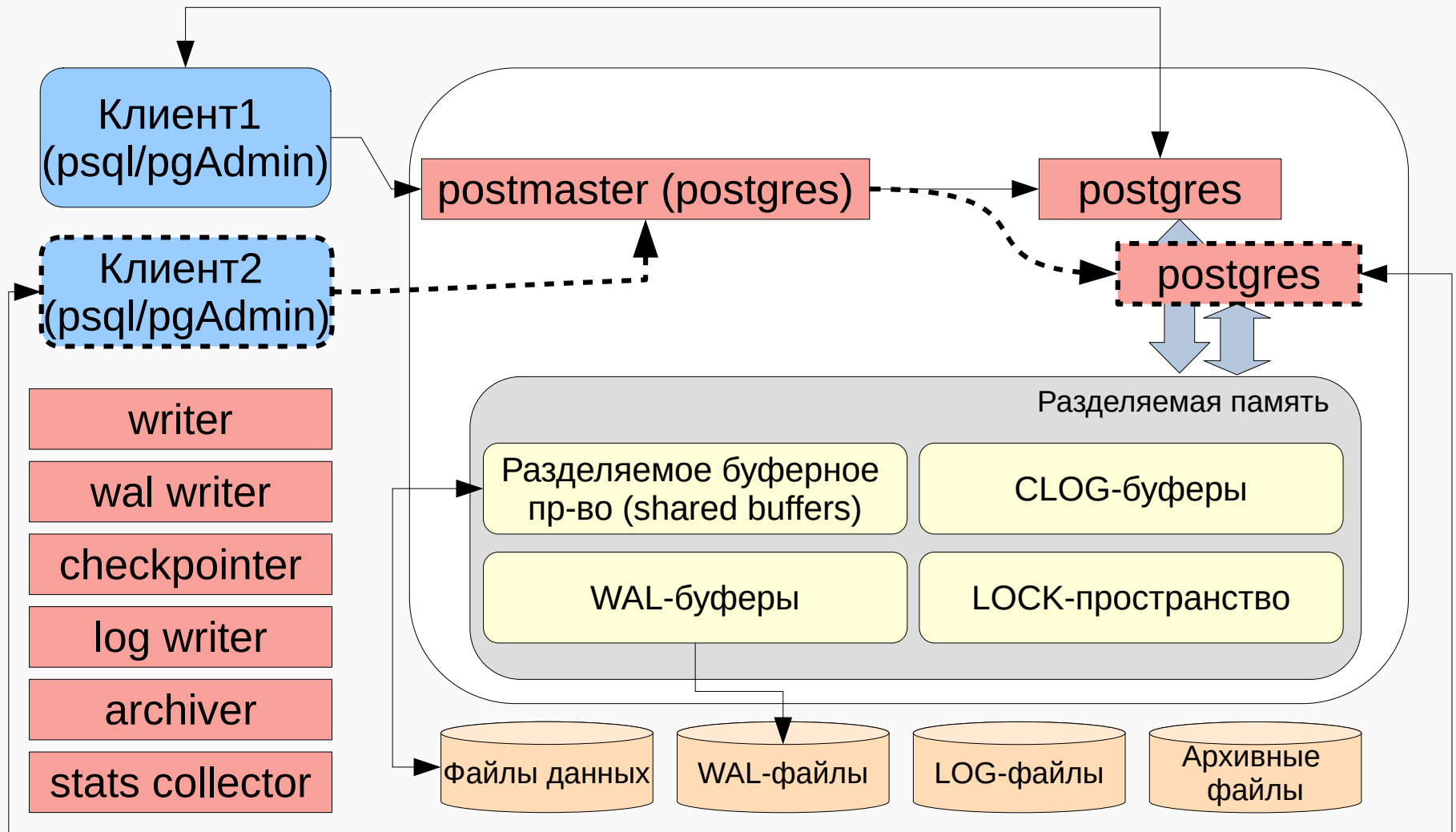
<https://www.interdb.jp/pg/index.html>

Лекция 8

Лектор:
Николаев В.В.

1. WAL в PostgreSQL

Архитектура PostgreSQL



WAL buffers

WAL — Write Ahead Log:

- Хранит информацию об изменениях данных в БД — WAL-записи (XLOG).
- Эта информация используется для воссоздания актуального состояния данных в случае восстановления базы данных (например, после сбоя).
- Настраивается через параметр `wal_buffers` — размер wal-буфера:
 - по умолчанию $1/32$ от разделяемого буфера;

WAL-записи

- WAL-записи фиксируют различные изменения состояния данных в БД:
 - INSERT, UPDATE, DELETE, COMMIT, ...;
- WAL-записи записываются в:
 - Write Ahead Log;
 - При операции COMMIT — происходит синхронизация с WAL (логом транзакций);
- WAL — общий для всего кластера.

PostgreSQL: WAL-записи

- Поддерживается REDO-лог.
- Запись об изменении попадает в журнал (на диск) раньше, чем данные на диск и clog.
- PGDATA/pg_wal:
 - журнал разбит на сегменты;
 - wal-segsize — размер сегмента;
 - wal-segsize можно изменить только при инициализации кластера PostgreSQL;

Особенности работы журнала

- В журнал не попадают:
 - действия над временными таблицами;
 - действия над UNLOGGED-таблицами;
 - **до** версии 10: хэш-индексы.
- UNLOGGED TABLE: изменения для данных в такой таблице не хранятся в WAL:
 - CREATE **UNLOGGED** TABLE Students ...

UNLOGGED можно использовать, **только** когда данные в таблице **не важны** (не страшны потери), и при этом требуется минимизировать число обменов IO

Основные составляющие записи в журнале:

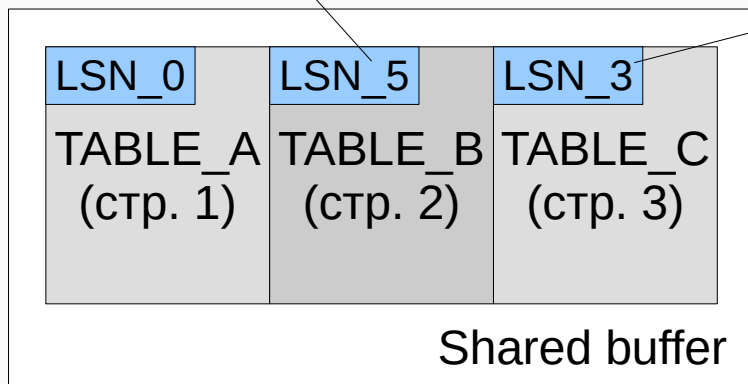
- Заголовок:
 - размер всей записи;
 - номер транзакции;
 - размер данных;
 - ссылка на предыдущую запись;
 - контрольная сумма;
- Данные об изменении — могут быть по-разному представлены.

WAL-записи

- У каждой WAL-записи есть **LSN (Log Sequence Number)** — уникальный идентификатор WAL-записи.
- Используется для нахождения позиции записи в WAL:
 - LSN — смещение до записи от начала журнала;
- LSN последней WAL-записи, относящейся к некоторой странице хранится в заголовке страницы.
- Для определения того, актуальна или нет некоторая страница (например, на момент восстановления) её LSN сравнивается с LSN в журнале.

WAL-записи

У страниц есть заголовок.



В заголовке параметр `pd_lsn`:
последний LSN с модификацией в этой странице

Восстановление

- WAL (**REDO-лог**) — журнал последовательно читается и перевыполняются операции:
 - для определения записей, которые нужно заново «выполнить» используется LSN.
- Чтобы сократить число записей для просмотра: используются **контрольные точки**:
 - процесс **checkpoint**;
 - начало последней контрольной точки — REDO point.

wal_level

Параметр позволяет задать уровень детализации операций в WAL:

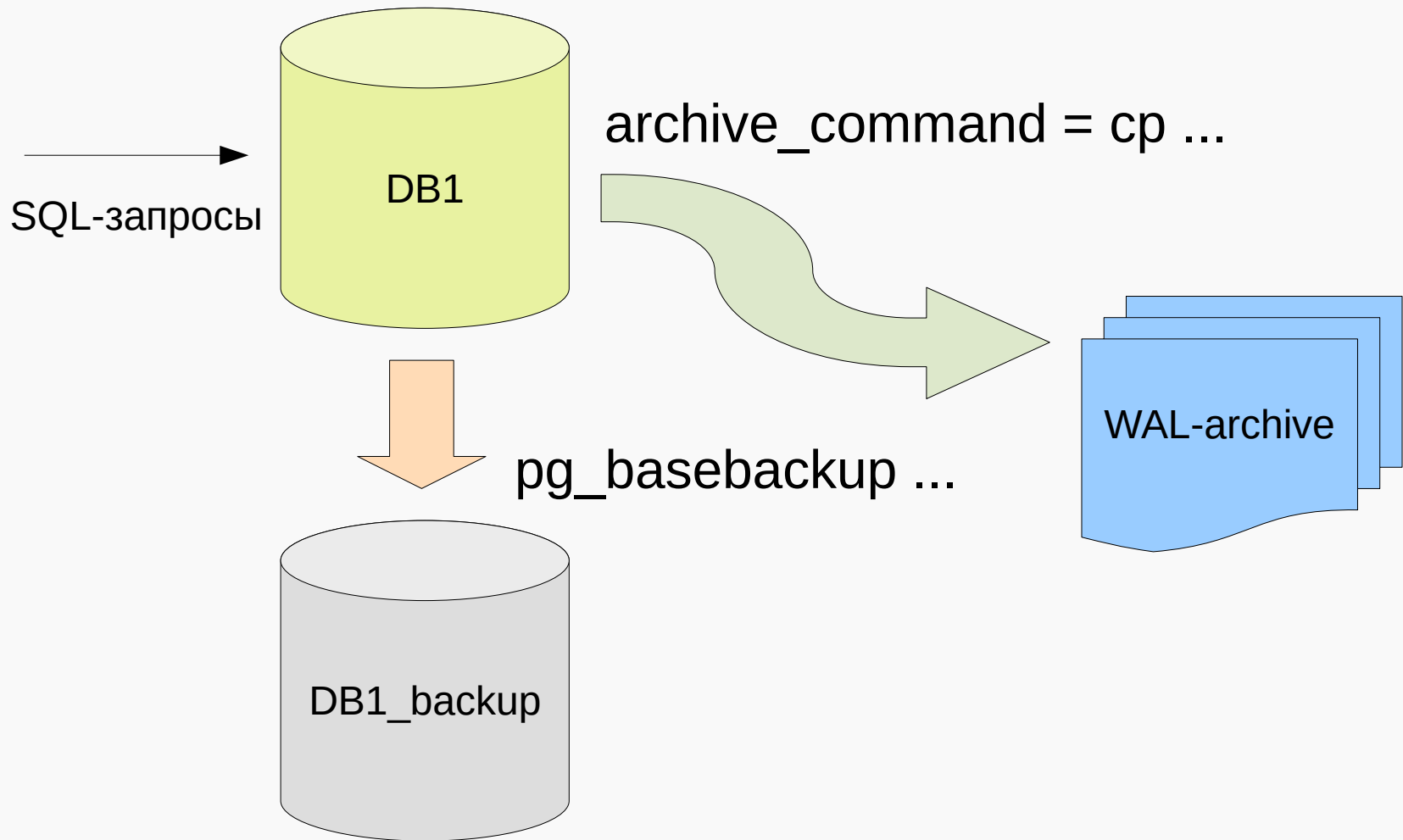
- **minimal** — только для восстановления после сбоя или immediate shutdown;
- **replica** — minimal + поддержка архивирования WAL и физической репликации;
- **logical** — replica + поддержка логической репликации (логического декодирования WAL);

Непрерывное архивирование

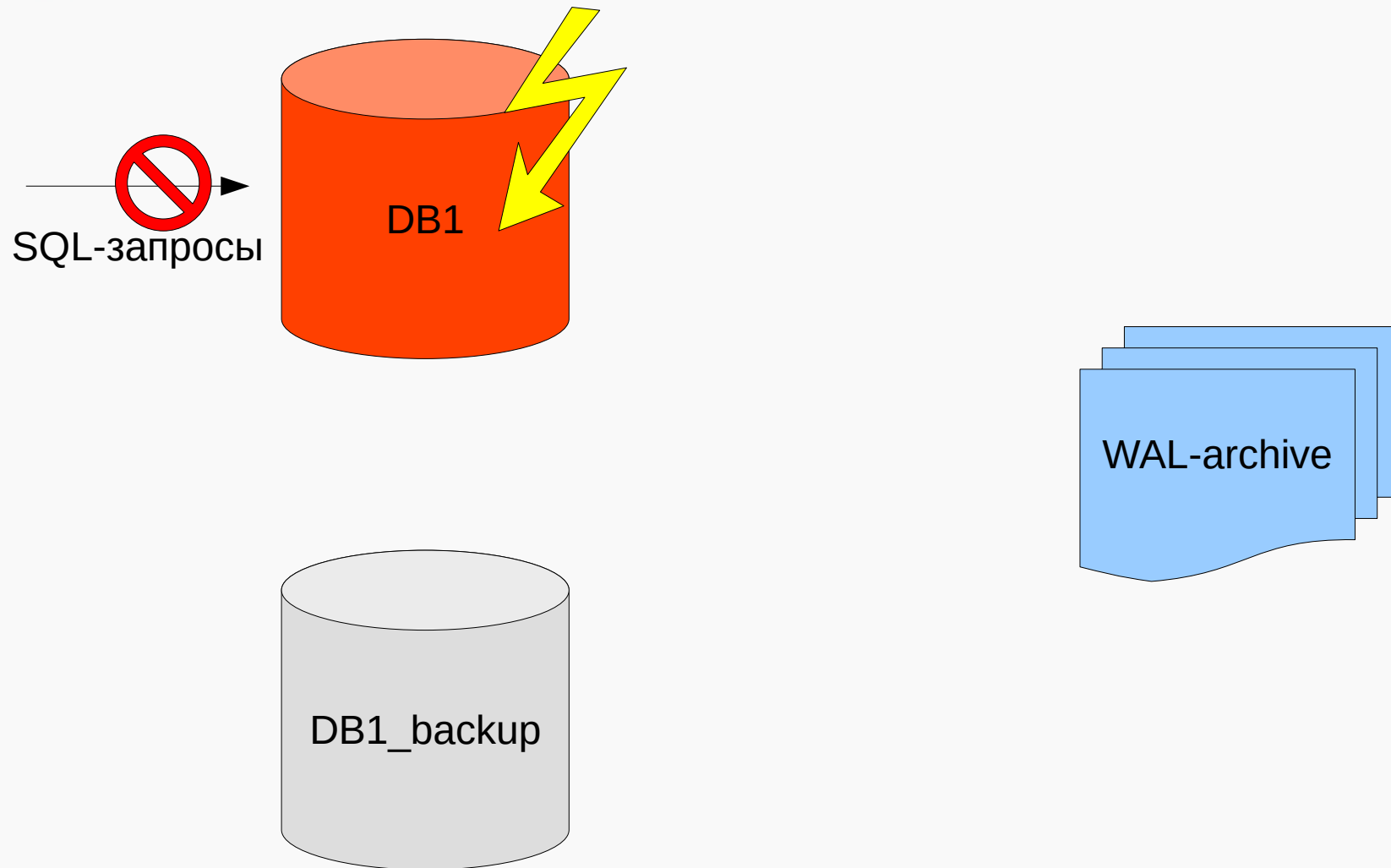
- Базируется на существовании WAL (PGDATA/pg_wal):
 - состояние БД может быть восстановлено путем повтора зафиксированных в логе операций;
 - конфигурационные файлы не восстанавливаются через WAL (postgresql.conf, ...);

Резервная копия = полная копия + WAL-файлы

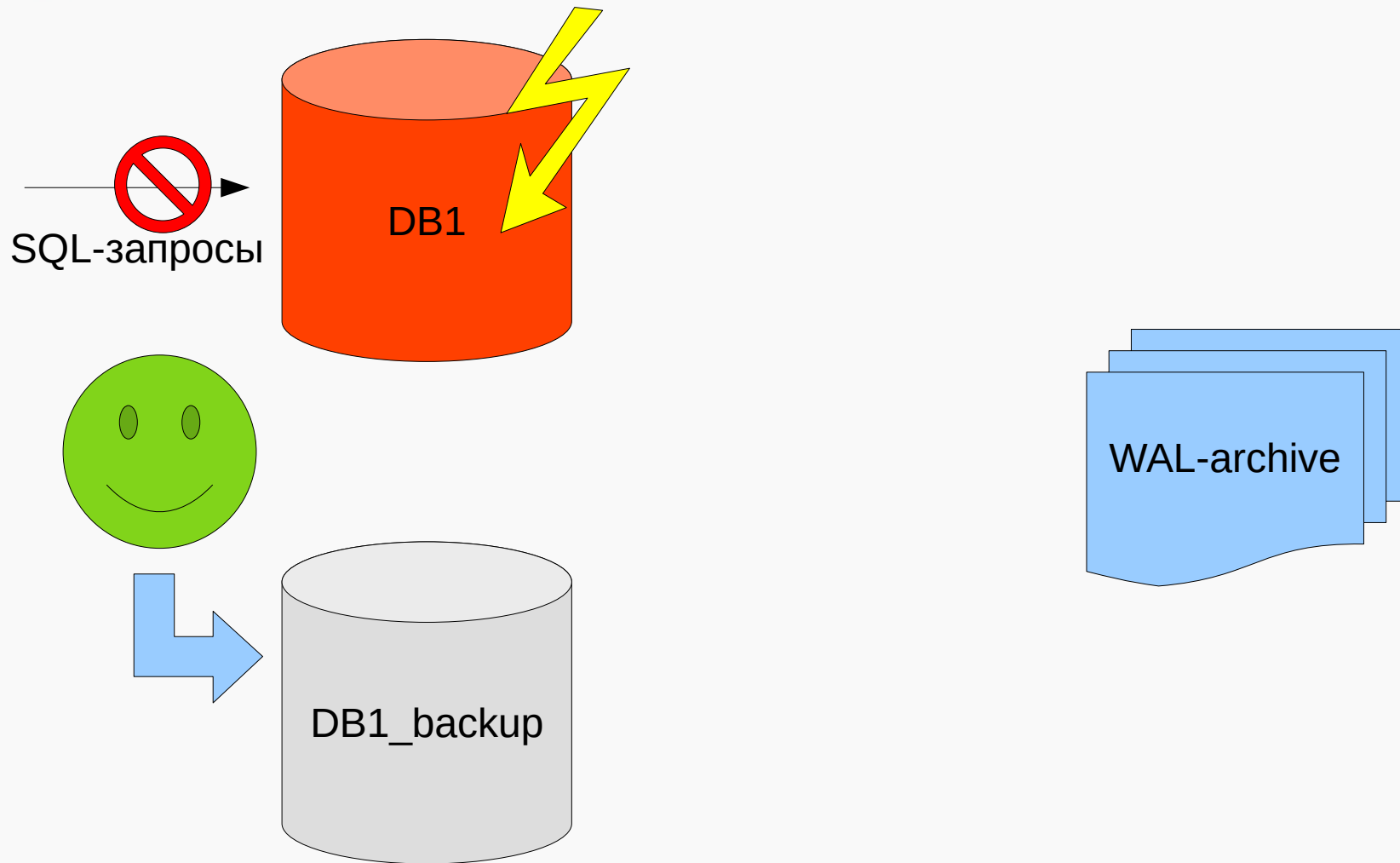
1. Нормальный процесс работы при непрерывном архивировании



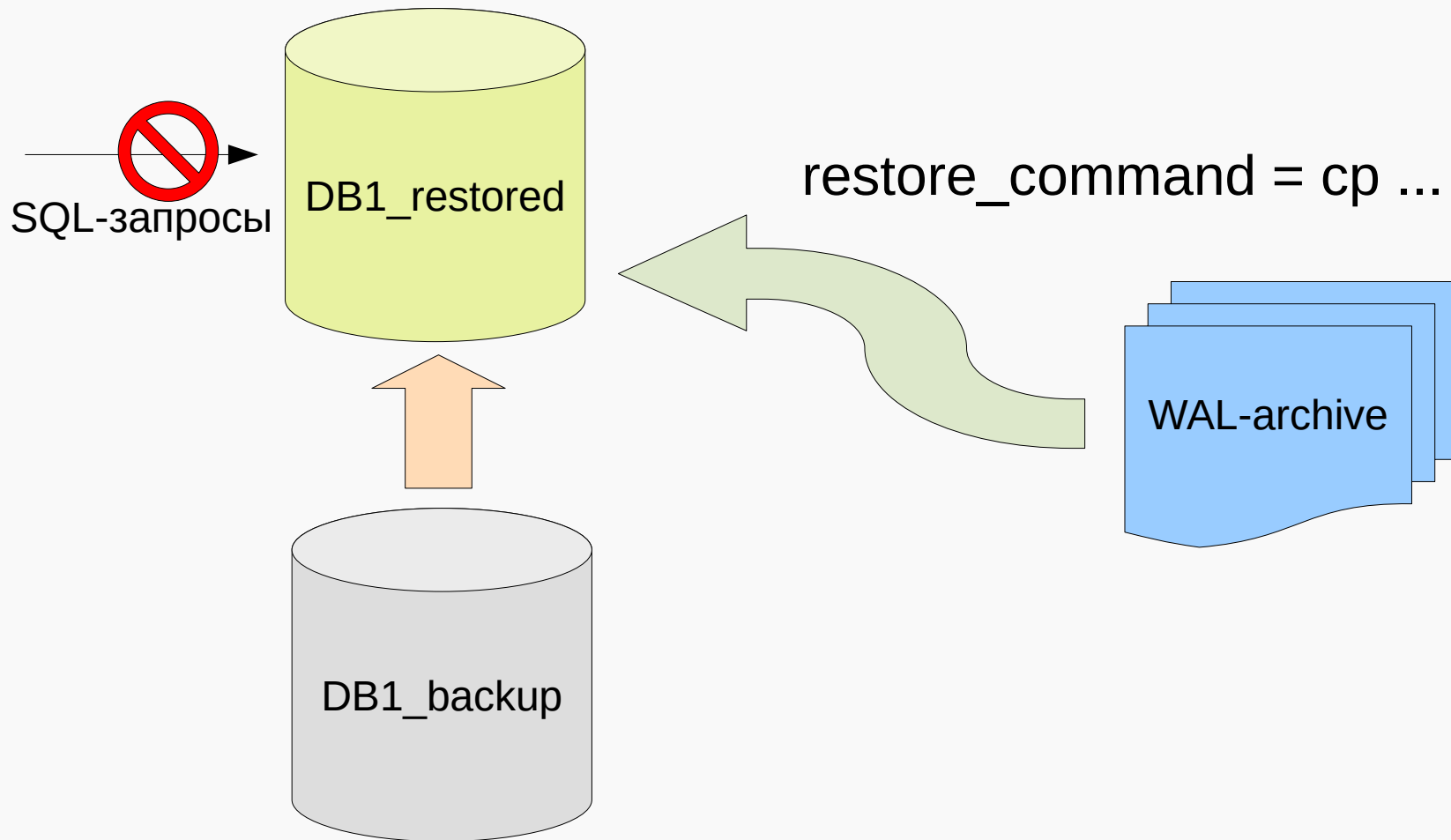
2. Сбой



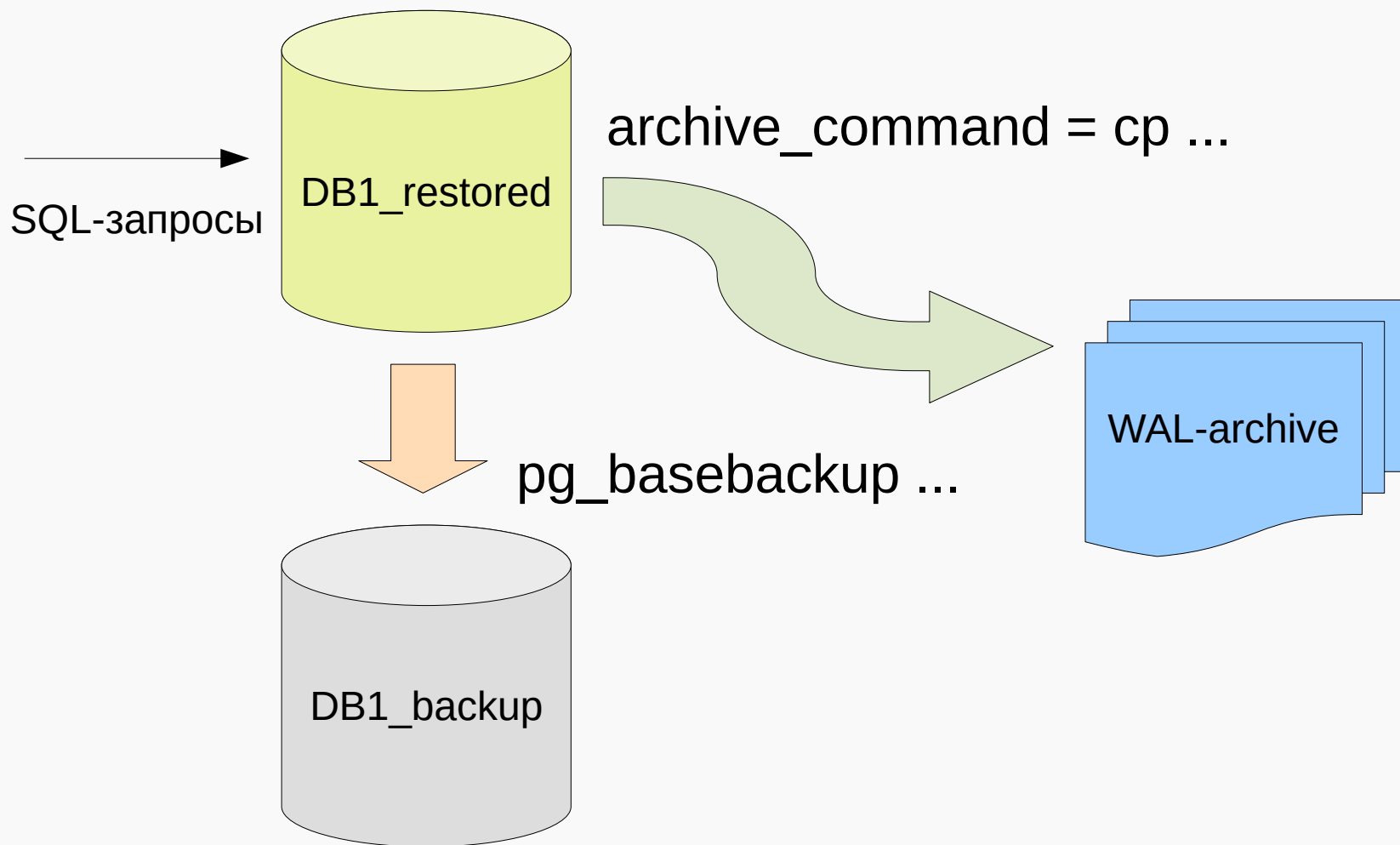
3. Обнаружение проблемы



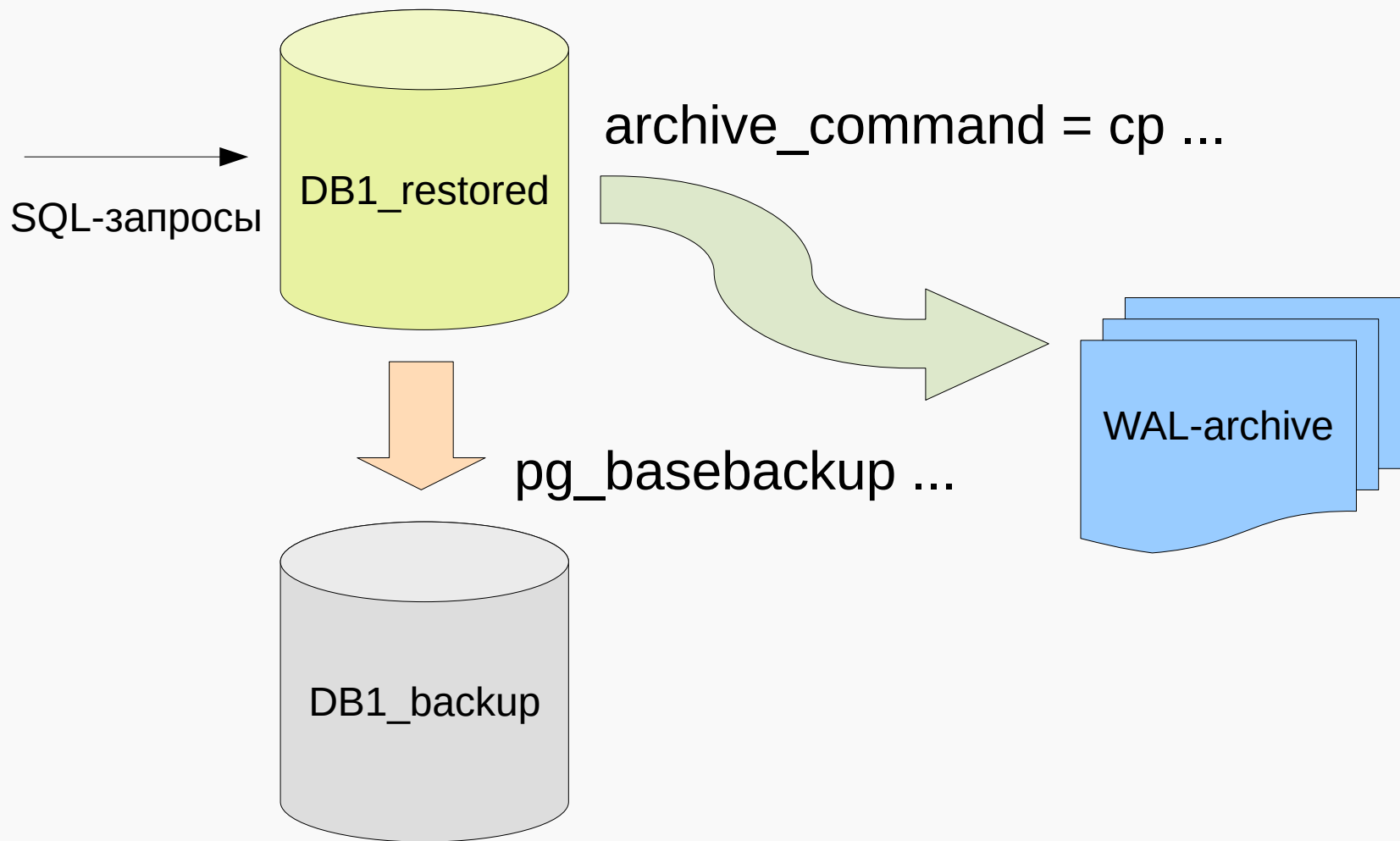
4. Восстановление



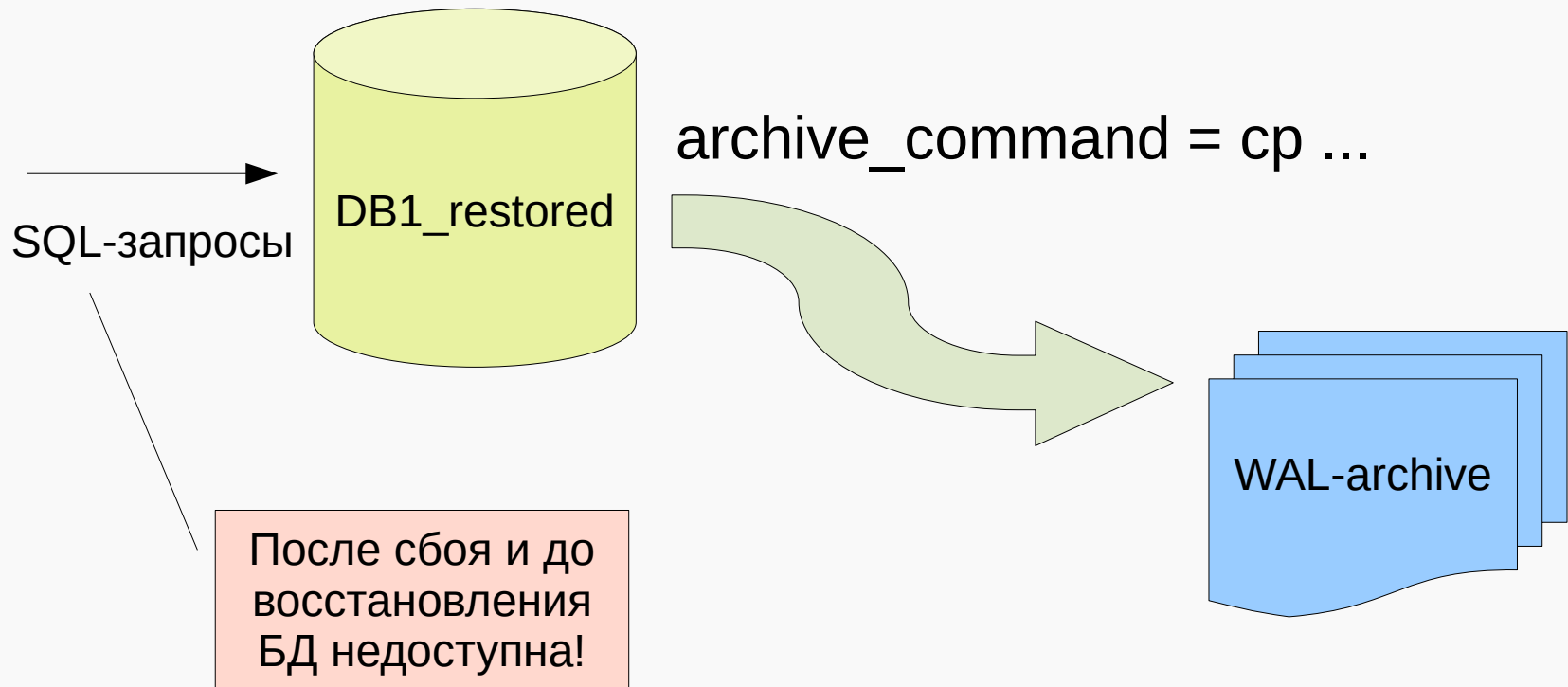
5. Состояние восстановлено



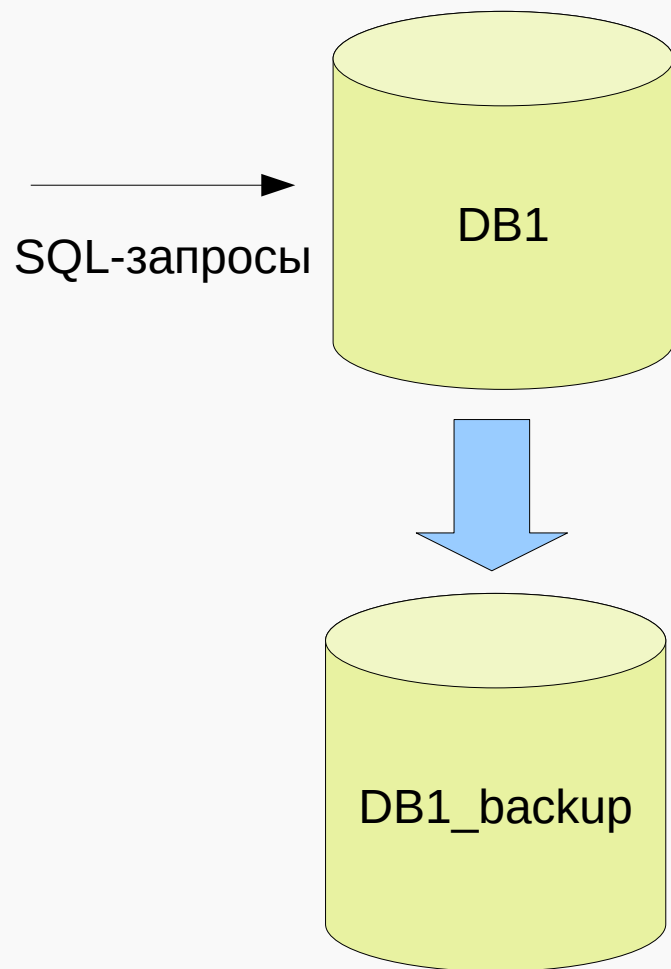
В чем проблема?



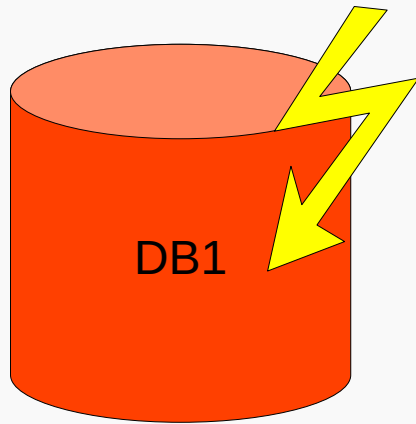
В чем проблема?



Возможное решение



Возможное решение:



→
SQL-запросы



2. Репликация данных в PostgreSQL

ЧТО МЫ ХОТИМ?

Высокий уровень доступности — (high availability) способность системы работать без сбоев (непрерывно) в течение определенного периода времени.

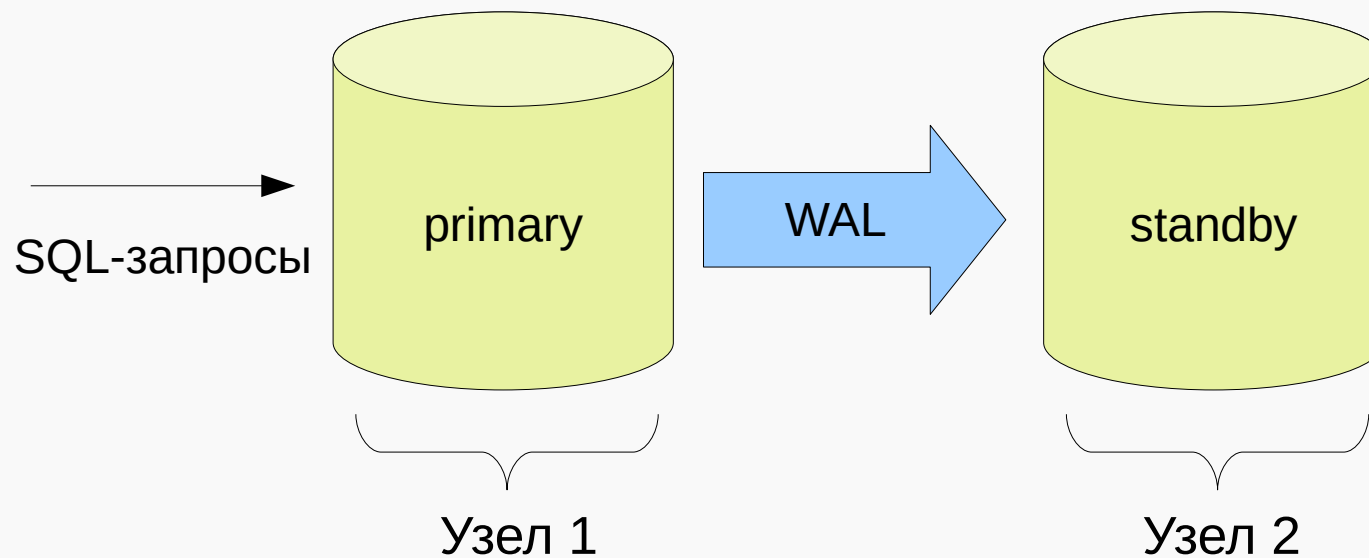
Как обеспечить высокий уровень доступности:

- обеспечить избыточность важных частей системы;
 - репликация данных.
- Failover, switchover.

Репликация

- Репликация данных — средство обеспечения избыточности. Заключается в сохранении и поддержании нескольких копий данных.
- Репликация в СУБД:
 - **Узлы** — отдельные серверы БД, обеспечивающие работу. Совокупность узлов — **кластер**.
 - У каждого узла — своя БД (оригинал или копия, реплика). Копии данных синхронизируются в реальном времени.
 - Один из серверов БД — главный (**master, primary**). Режим — чтение/запись. Если несколько master-узлов — multimaster-репликация.
 - Остальные узлы — зависимые (**slave, standby**). Если режим — только чтение — **hot standby**.

Репликация



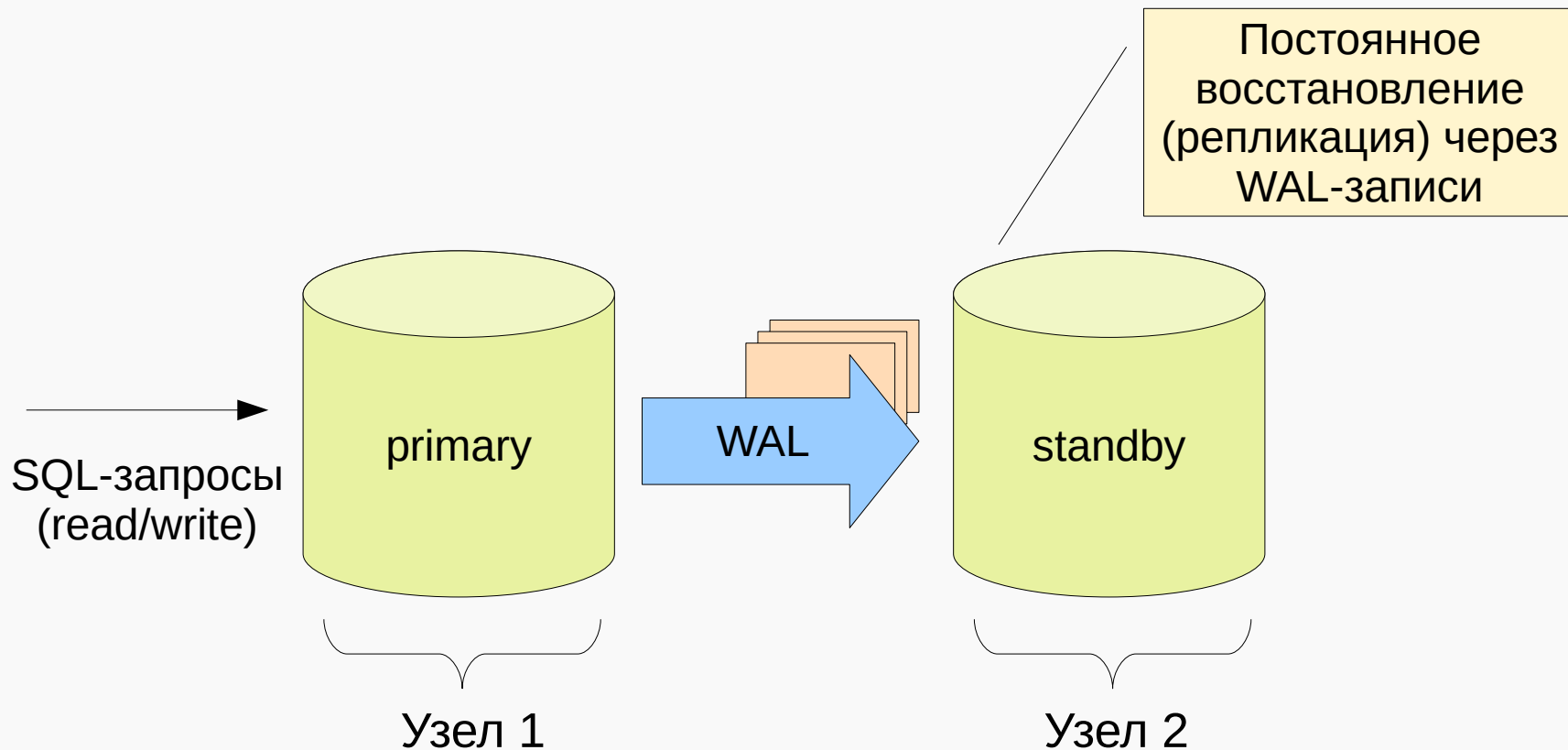
Репликация в PostgreSQL

- В PostgreSQL репликация может быть:
 - **Физическая** — пересылаются файлы (WAL). standby — содержит копию primary:
 - ✓ **асинхронная** — данные для синхронизации состояний серверов БД пересылаются без подтверждения получения;
 - ✓ **синхронная** — пересылки данных идут с подтверждением.
 - **Логическая** — пересылаются декодированные из WAL операции.

Физическая репликация

- **Физическая репликация** — пересылаются файлы (WAL). Standby — содержит копию primary.
- Требуется — `wal_level = replica` (по умолчанию).
- Поточковая репликация (**streaming replication**) — WAL-записи передаются по соединению между primary и standby:
 - Использует свой протокол для отправки WAL-файлов — WAL-sender, WAL-receiver.
- standby — осуществляет восстановление на основе полученных WAL для поддержки актуального состояния.

Потоковая репликация



Физическая репликация в PostgreSQL

- Пользователь для репликации:

```
CREATE USER PHYSREPL WITH REPLICATION ...
```

- pg_hba.conf: host replication physrepl all md5
- backup: pg_basebackup -d 'conn params' -D 'path'

} primary

- standby.signal в PGDATA.

- postgresql.conf:

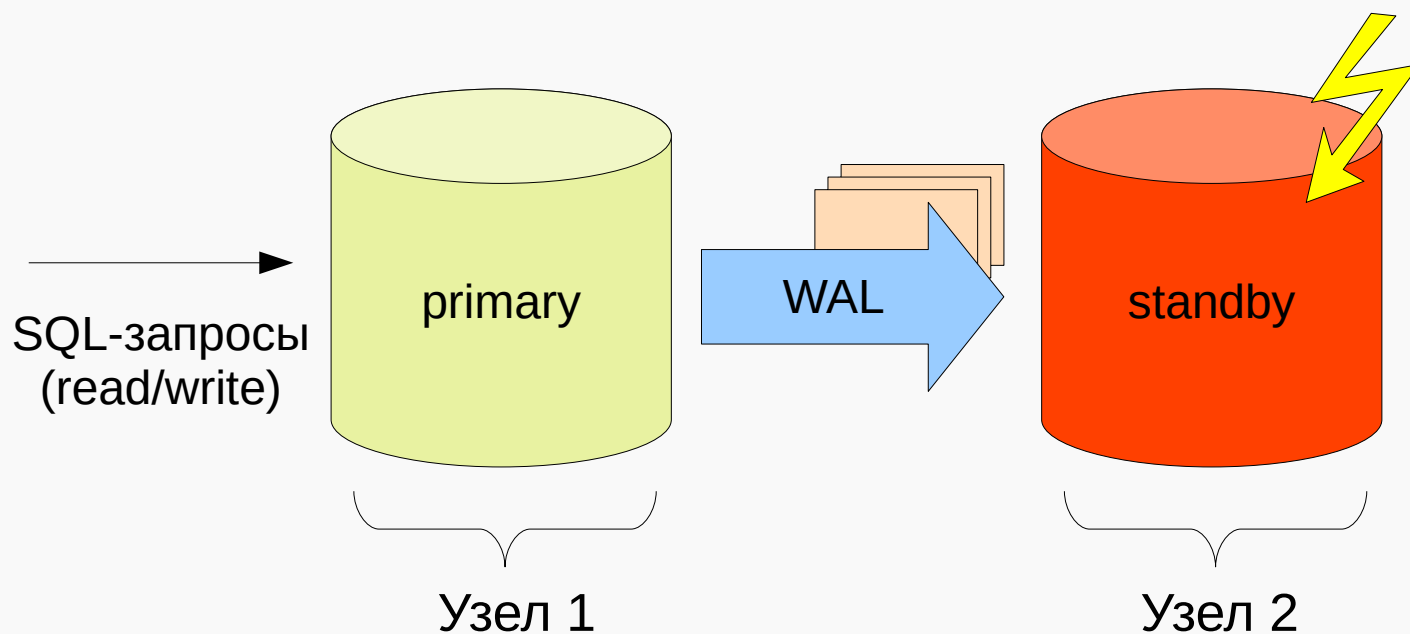
```
primary_conninfo = 'host=... user=physrepl'
```

} standby

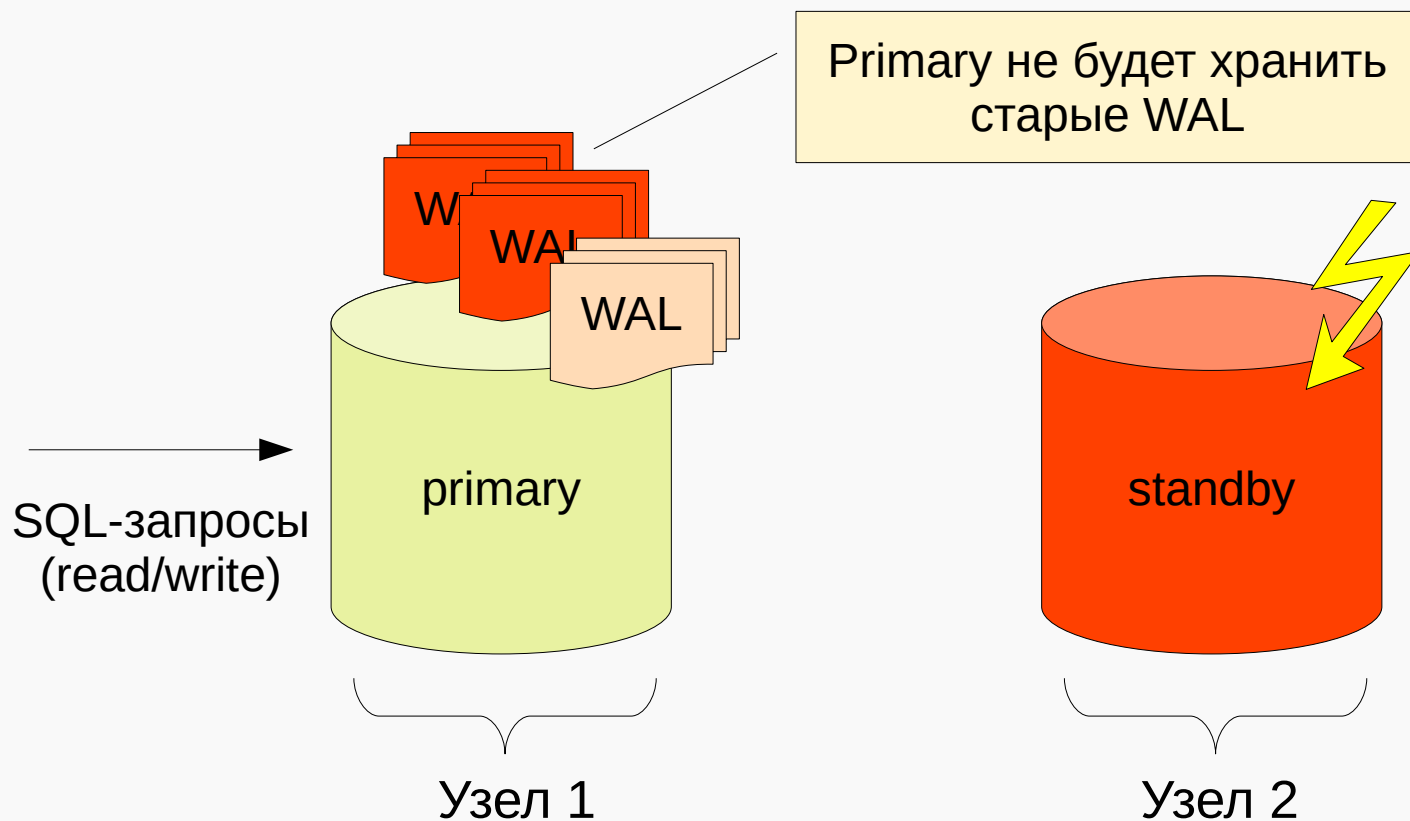
standby.signal

- Файл в PGDATA.
- Позволяет запустить сервер в режиме standby.
- Если установлен:
 - После обработки архивированных в данный момент WAL, сервер продолжает пытаться получить новые WAL-сегменты через:
 - ✓ `restore_command`
 - ✓ WAL-записи с мастера, используя `primary_conninfo`
- `primary_conninfo` — используется только, если `standby.signal` установлен.

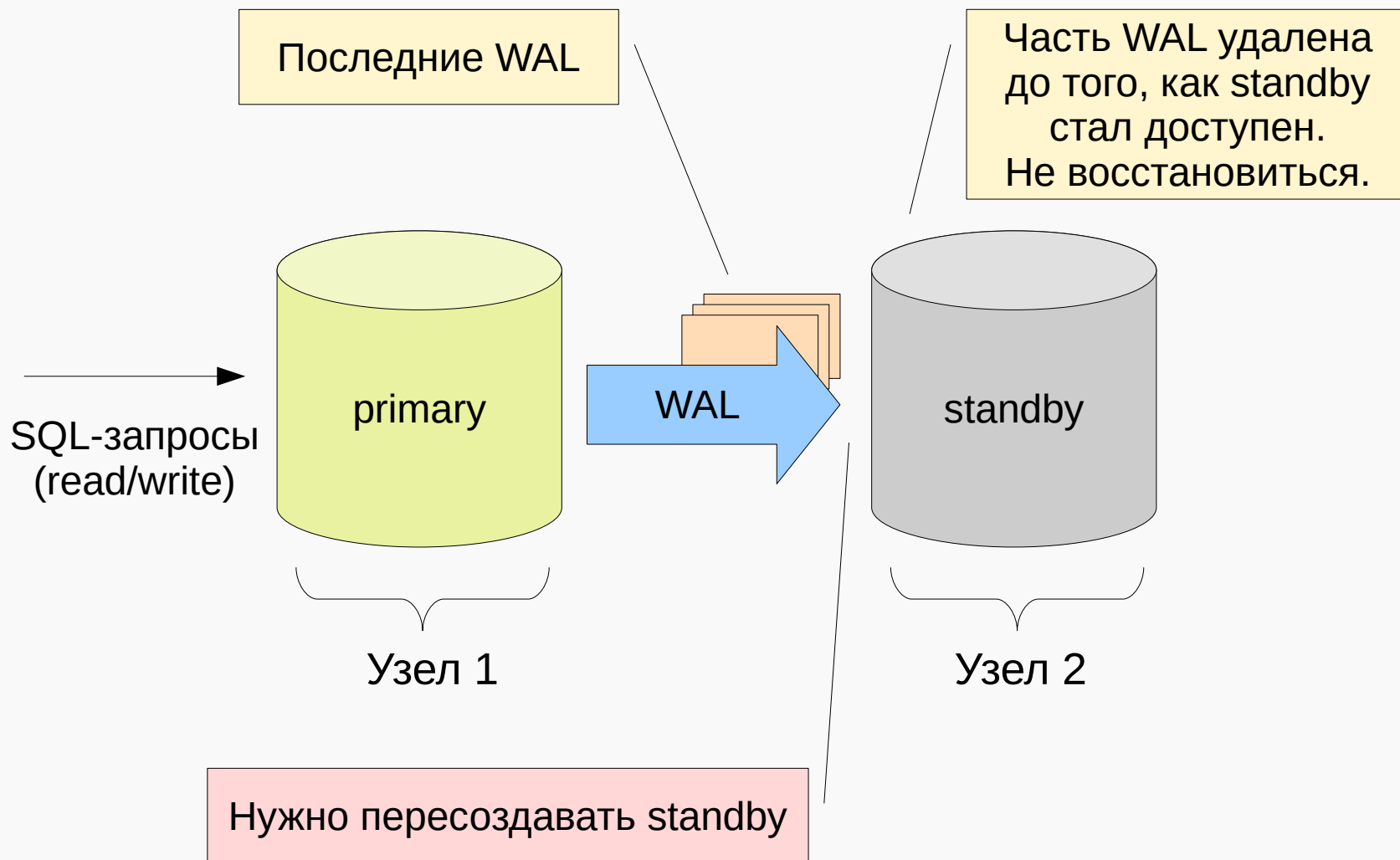
Что если произошла временная проблема со standby?



Что если произошла временная проблема со standby?



Что если произошла временная проблема со standby?



wal_keep_size

- Задается в postgresql.conf.
- Определяет минимальное количество ненужных (для мастера) WAL-сегментов, которые должны храниться.
- По умолчанию — 0.
- Эти WAL-сегменты будут доступны standby:
 - **НЕТ** гарантии, что для standby будет достаточно этих сегментов.

- Позволяет хранить сегменты, пока они не отправлены standby.
- Таких сегментов может много накопиться (если standby не доступен).
- **max_slot_wal_keep_size** — позволяет ограничить число хранимых сегментов.
- Для использования:
 - нужно создать слот: `SELECT * FROM pg_create_physical_replication_slot('node2_slot');`
 - на standby: **primary_slot_name** = 'node2_slot'.

Полезные параметры/команды

- `--max_wal_senders` — число standby + 1. Если используется `--wal_method=stream` — еще +1 — для backup.
 - Определяет число процессов для отправки WAL (WAL sender).
- `--wal_method=stream` — при использовании `pg_basebackup` будет открыто второе соединение для передачи WAL параллельно с созданием копии.
 - Используется по умолчанию.
- Просмотр состояния реплики:
 - представление `pg_stat_replication`;

Синхронная репликация

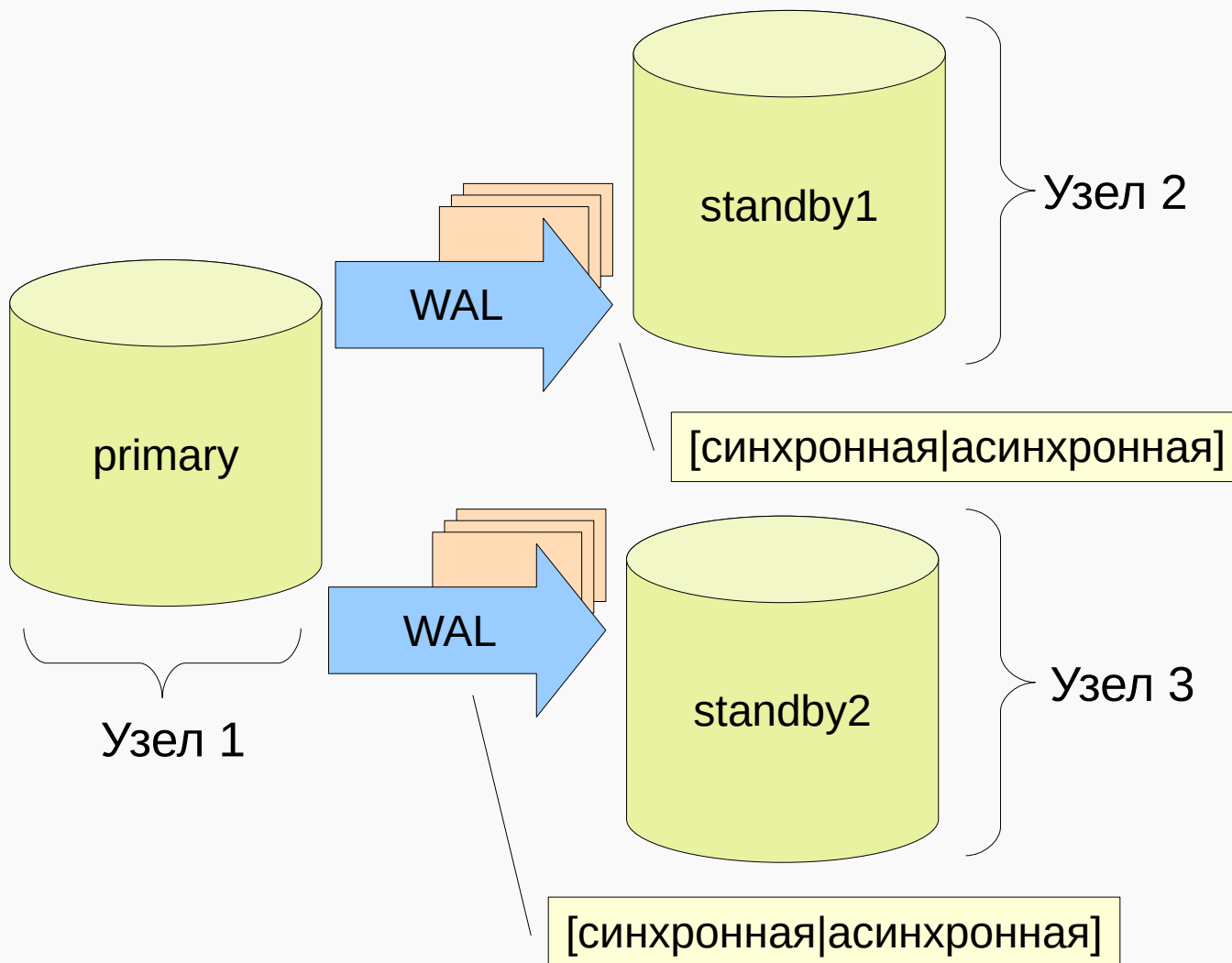
- Определяет, нужно ли ждать обработки WAL на репликах, когда происходит COMMIT транзакции.
- параметр `synchronous_commit` (default = on) + `synchronous_standby_names`:
 - по умолчанию включена асинхронная репликация.
- Дает возможность подтвердить, что транзакция реплицирована хотя бы на одном standby.
- Указать список standby серверов с синхронной репликацией:

```
synchronous_standby_names = 'node2, node3, node4'
```

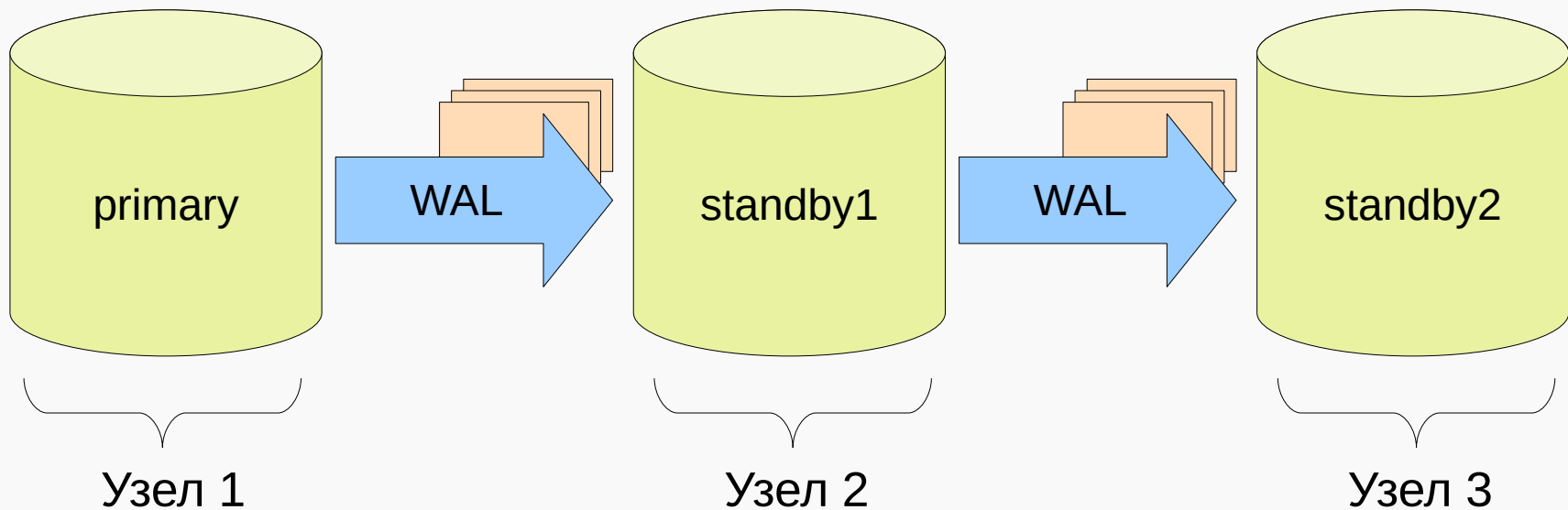
Synchronous commit

	Долгов-ть локального коммита	Долгов-ть сбой БД	Долгов-ть сбой ОС	standby - согл-ть запросов
remote_apply	+	+	+	+
on	+	+	+	
remote_write	+	+		
local	+			
off				

Репликация на несколько узлов



Ступенчатая репликация: cascading replication



Особенности standby

- **warm standby** — режим standby, при котором логи (WAL), архивированные primary-сервером, собираются standby и происходит непрерывное восстановление состояния БД по полученным WAL. Warm standby не позволяет осуществлять запросы.
- **hot standby** — сервер работает в режиме чтения — кроме восстановления по WAL, позволяет осуществлять запросы к данным (read-only).

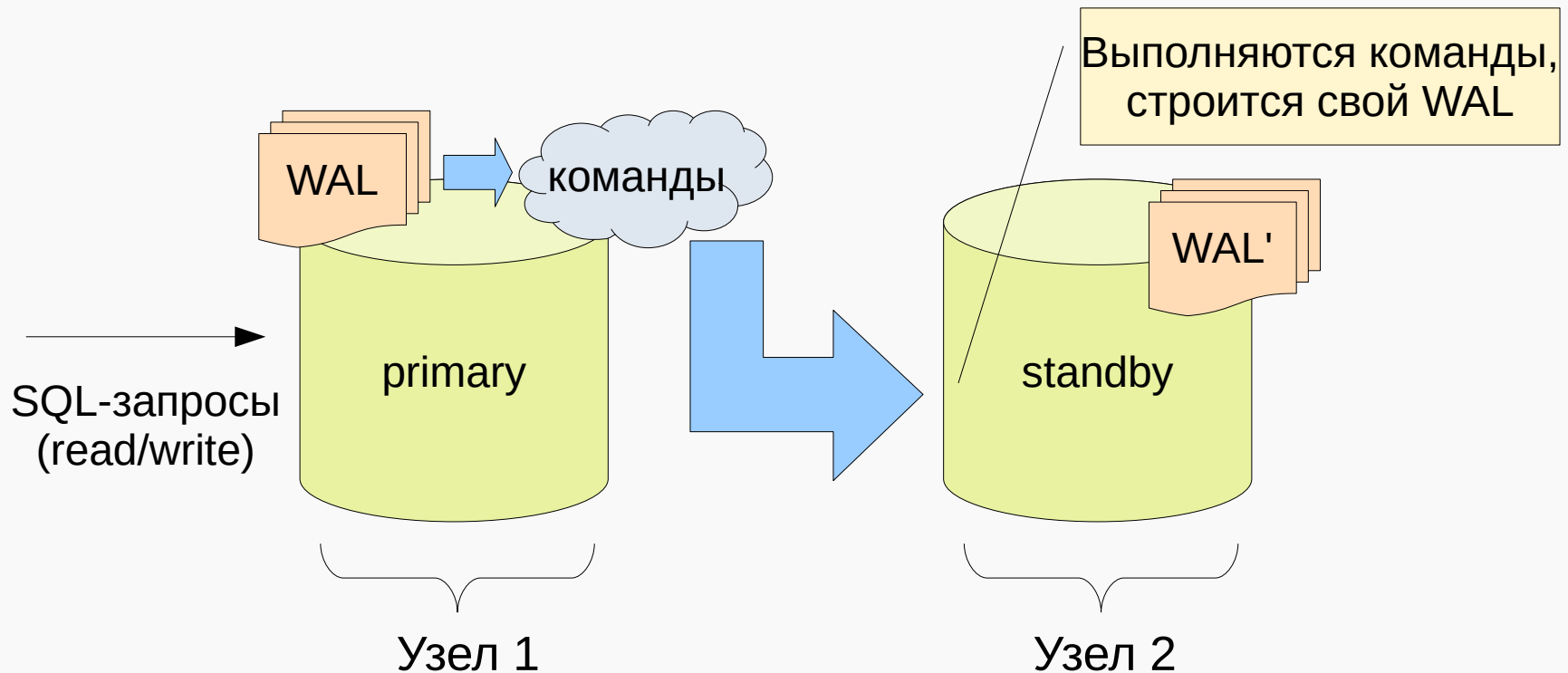
Параметр `hot_standby = true` (по умолчанию).

3. Логическая репликация данных в PostgreSQL

Логическая репликация

- **Логическая репликация** — пересылаются команды, реконструированные из WAL.
- Требуется — `wal_level = logical`.
- `standby` — осуществляет восстановление на основе полученных команд для поддержки актуального состояния.
- Используется модель «публикаций» «подписок»:
 - после логического декодирования команды публикуются на мастере;
 - опубликованные команды могут получить подписанные `standby`.

Логическая репликация



Логическая репликация в PostgreSQL (master)

- wal_level = logical
- Пользователь для репликации (master):
`CREATE USER LOGICREPL WITH REPLICATION ...`
- pg_hba.conf: host replication logicrepl all md5.
- Выдать права для LOGICREPL на нужные объекты.
- `CREATE PUBLICATION all_t_publ FOR ALL TABLES;`

primary

Логическая репликация в PostgreSQL

- wal_level = logical
- Воссоздать структуру таблиц.
- CREATE SUBSCRIPTION all_t_subscr
CONNECTION 'user=logicropl ... dbname=db_name'
PUBLICATION all_t_publ;

standby

Особенности

- Можно реплицировать часть БД (отдельный набор таблиц).
- Не поддерживается репликация DDL.
- Возможна репликация даже для узлов с разными версиями PostgreSQL.
- Standby может работать не в read-only режиме.

Документация PostgreSQL:

<https://www.postgresql.org/docs/14/index.html>

Лицензия PostgreSQL:

PostgreSQL is released under the PostgreSQL License, a liberal Open Source license, similar to the BSD or MIT licenses.

PostgreSQL Database Management System
(formerly known as Postgres, then as Postgres95)

Portions Copyright © 1996-2022, The PostgreSQL Global Development Group

Portions Copyright © 1994, The Regents of the University of California

Permission to use, copy, modify, and distribute this software and its documentation for any purpose, without fee, and without a written agreement is hereby granted, provided that the above copyright notice and this paragraph and the following two paragraphs appear in all copies.

IN NO EVENT SHALL THE UNIVERSITY OF CALIFORNIA BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, INCLUDING LOST PROFITS, ARISING OUT OF THE USE OF THIS SOFTWARE AND ITS DOCUMENTATION, EVEN IF THE UNIVERSITY OF CALIFORNIA HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE UNIVERSITY OF CALIFORNIA SPECIFICALLY DISCLAIMS ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE SOFTWARE PROVIDED HEREUNDER IS ON AN "AS IS" BASIS, AND THE UNIVERSITY OF CALIFORNIA HAS NO OBLIGATIONS TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

<https://www.interdb.jp/pg/index.html>